

Queueing Theory for Disaggregated LLM Serving

Prefill instances, decode instances and the KV cache between them

Lecture notes for a five-lecture course

Serving Queue Theory Project

September 2026

Contents

About These Notes	3
1 Probabilistic Foundations: Arrivals, Service, and Little's Law	5
1.1 Queueing systems and performance measures	5
1.2 From a serving system to a queueing model	6
1.2.1 A language for serving deployments	7
1.2.2 The disaggregated replica	11
1.2.3 What the program implies	13
1.3 The exponential distribution and memorylessness	15
1.4 The Poisson process	16
1.5 PASTA: Poisson arrivals see time averages	17
1.6 Little's law	19
1.7 Exercises	20
2 Markovian Queues: Memory Slots, Processor Sharing, and Insensitivity	21
2.1 Continuous-time Markov chains	21
2.2 Birth-death processes and M/M/1	22
2.3 Multi-server queues and memory slots	23
2.4 Processor sharing and state-dependent capacity	24
2.5 Insensitivity	27
2.6 The price at a processor-sharing station	28
2.7 Exercises	29
3 The Prefill Instance as an M/G/1 Queue: the Price of a Miss	30
3.1 The M/G/1 queue and the residual service time	30
3.2 The Pollaczek–Khinchine formula	31
3.3 Mixtures of hits and misses	33
3.4 The price of a miss	33
3.5 Exercises	36
4 Closed Systems and Memory: Finite Populations, MVA, Campbell's Theorem, Eviction	37
4.1 Closed systems and mean value analysis	37
4.2 M/G/ ∞ and Campbell's theorem	40
4.3 Eviction: the covering knapsack problem	41
4.4 Exercises	43
5 Putting It Together: A Queueing Model of Disaggregated Serving	44
5.1 The model on one page	44
5.2 Where the variance of prefill work comes from	45
5.3 Three prices	45
5.4 The decision problem with two pools	46
5.5 How long to keep	47
5.6 How many sessions to admit	49
5.7 Misses that feed themselves	49
5.8 What is exact and what is approximate	51

5.9 Exercises	51
-------------------------	----

About These Notes

Goal. These notes accompany a short course for readers who know real analysis and mathematical statistics but have not met queueing theory. They build, from definitions, the queueing tools needed to model one architecture of large language model (LLM) serving, *prefill-decode disaggregation*, and to prove the results a scheduler for it needs. Measure-theoretic probability is not needed; we use random variables, expectation, conditional expectation, the strong law of large numbers and power series freely.

The system. An LLM request is served in two phases. *Prefill* reads the whole prompt in one pass and produces the first token; the time from the request's arrival to its first token is the *time to first token* (TTFT). *Decode* then produces one token per step until the answer is complete. Both phases need the attention state of every prompt token, the *KV cache*. In a disaggregated deployment the two phases run on different machines:

- *prefill instances* run prefills only, one after another; they keep a *prefix cache* of recent prompts, so that a request that starts with a cached prefix (a *hit*) computes only its new tokens, while one whose prefix was evicted (a *miss*) recomputes its whole context;
- a *link* carries each request's KV cache from its prefill instance to a decode instance;
- *decode instances* run decodes only, advancing all their running requests by one token per step.

Both kinds of instance hold KV in a memory pool of fixed size, so a request is admitted to an instance only when its context fits. An *agent* is a session of many such requests (*turns*): after each turn it calls a tool, pauses, and sends the next turn, whose prompt is the previous one with the tool's output appended. Requests arrive at random, wait for memory and for servers, receive service and leave: this is the object of queueing theory.

Why disaggregation makes the queueing model clean. When prefill and decode share a machine, each slows the other and the queue each one sees depends on the other's load. Separated, each instance is a single, recognisable queue: a first-in first-out server for prefill, a processor-sharing batch for decode, a shared-bandwidth link in between, and a multi-server memory station in front of each instance. The notes study these pieces one at a time and then put them together.

Map. What the course proves and where:

Question	Tools and results
How long does a request wait for memory?	birth–death processes (Theorem 2.6), M/M/ m and the Erlang C formula (Lecture 2)
Does the decode batch care about the length distribution?	processor sharing and insensitivity (Theorem 2.16), Theorem 2.14
What does one prefix miss cost the prefill instance?	PASTA (Theorem 1.23), residual service time (Lemma 3.3), Pollaczek–Khinchine (Theorem 3.4), the price of a miss (Theorem 3.10)
What does it cost the link and the decode instance?	the price at a processor-sharing station (Proposition 2.18)
How many sessions can be live?	closed systems and mean value analysis (Proposition 4.6), Campbell’s theorem (Theorem 4.13)
What should each instance keep, and for how long?	covering knapsack and the threshold rule (Theorem 4.16), price-blind keys (Proposition 4.17), Lecture 5
Can misses feed themselves?	monotone fixed points (Lecture 5)

Plan. Lecture 1 covers arrival processes, Little’s law and the queueing model of the disaggregated system; Lecture 2 Markovian queues, memory slots and processor sharing; Lecture 3 the prefill instance as an M/G/1 queue and the price of a miss; Lecture 4 closed systems, memory occupancy and eviction; and Lecture 5 assembles the pieces into a model of the disaggregated system with its transfer overhead, prices and feedback. The exercises at the end of each lecture are meant to be solved before the next one. Good introductory textbooks are [12, 5]; [1] is the rigorous reference.

Lecture 1

Probabilistic Foundations: Arrivals, Service, and Little's Law

This lecture defines the vocabulary of queueing theory and proves three tools that every later lecture relies on: the memorylessness of the exponential distribution, the Poisson process together with PASTA, and Little's law. We assume undergraduate probability (conditional expectation, the strong law of large numbers, probability generating functions).

1.1 Queueing systems and performance measures

Definition 1.1 (Queueing system)

A **queueing system** consists of four elements.

1. **Customers** (or jobs): entities that arrive at the system, require some amount of processing, and leave. The arrival time of the k -th customer is a_k , numbered so that $0 \leq a_1 \leq a_2 \leq \dots$.
2. The **arrival process**: the sequence of arrival times $(a_k)_{k \geq 1}$, or equivalently the counting process $A(t) = \#\{k : a_k \leq t\}$.
3. A **server** and **service times**: the server is the resource that processes customers. Customer k needs time S_k when it has the server to itself. S_k is also called the **work** or service requirement of customer k .
4. A **waiting room** and a **service discipline**: the space where customers arriving at a busy server wait, and the rule that decides whom to serve, when, and how much.

Definition 1.2 (Service disciplines)

FIFO (first in, first out) serves one customer at a time, to completion, in order of arrival. **Processor sharing** (PS) serves all customers in the system simultaneously at equal rates; it is defined formally in Lecture 2. A discipline under which the server never idles while work is present is called **work-conserving**. Both FIFO and PS are work-conserving.

Definition 1.3 (Kendall notation)

A queueing system is written $A/S/c/K/N$. Here A is the distribution of the interarrival times, S the distribution of the service times, c the number of servers, K the system capacity (∞ if omitted), and N the size of the customer population (∞ if omitted). The distribution symbols are M (Markov: exponential and independent), D (deterministic: constant), and G (general: an arbitrary distribution, i.i.d.).

For example, $M/M/1$ has Poisson arrivals, exponential service and one server (Lecture 2); $M/G/1$ allows a general service distribution (Lecture 3); $M/M/1/N$ is a closed system with only N customers (Lecture 4); and $M/G/\infty$ is an infinite-server system with no waiting (Lecture 4).

Definition 1.4 (Performance measures)

Let d_k be the time at which customer k leaves the system.

1. The **sojourn time** (response time) is $W_k = d_k - a_k$.
2. The **waiting time** $W_{q,k}$ is the time from the arrival of customer k until its service first begins. For a FIFO single server, $W_k = W_{q,k} + S_k$.
3. The **number in system** is $N(t) = \#\{k : a_k \leq t < d_k\}$. The number in the waiting room is written $N_q(t)$.

Quantities in queueing theory are defined by two different kinds of averages, and the distinction will be essential several times.

Definition 1.5 (Time averages and customer averages)

The **time average** of the process $N(t)$ is $L = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t N(s) ds$, and the **customer average** of the sequence (W_k) is $W = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n W_k$ (when the limits exist). L_q and W_q are defined in the same way. If the limits are constants with probability one, then in the stationary regime they equal $\mathbb{E}[N]$ and $\mathbb{E}[W]$ respectively.

Definition 1.6 (Arrival rate, load, stability)

The **arrival rate** is $\lambda = \lim_{t \rightarrow \infty} A(t)/t$. The **load** (utilisation) of a single-server system with mean service time $\mathbb{E}[S]$ is

$$\rho = \lambda \mathbb{E}[S].$$

ρ is the amount of work arriving per unit time and is dimensionless. The system is **stable** if $N(t)$ has a proper limiting distribution (or at least $L < \infty$) and every customer leaves in finite time.

A single server processes at most one unit of work per unit time, so if $\rho > 1$ the unfinished work grows linearly and the system cannot be stable. For a work-conserving $G/G/1$ system (i.i.d. interarrival and service times) it is known that $\rho < 1$ is essentially necessary and sufficient for stability [15]. For the models in these notes we assume $\rho < 1$ as the stability condition and verify it model by model.

1.2 From a serving system to a queueing model

A queueing model of a serving system keeps what decides which request waits and for how long, and leaves out the rest. Figure 1.1 shows the model of a disaggregated deployment used throughout these notes. The queueing objects it uses (multi-server queues, processor sharing,

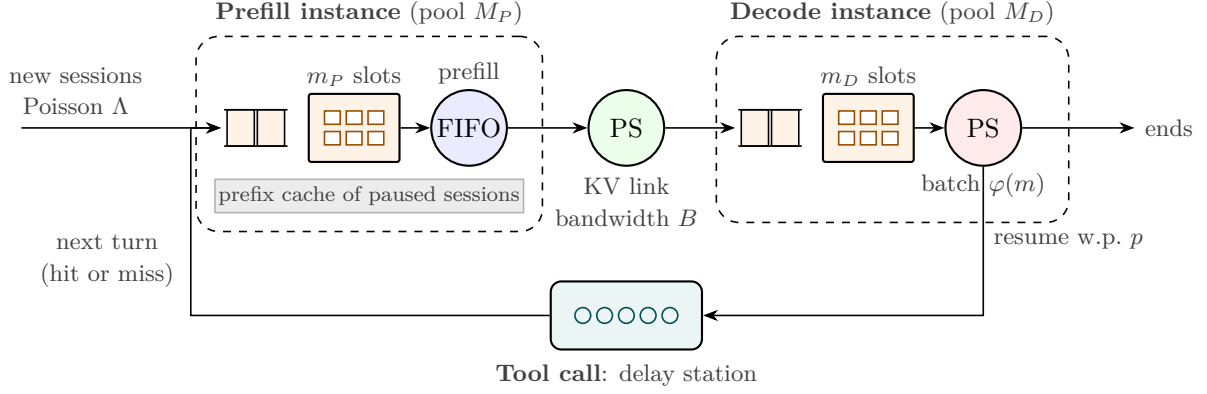


Figure 1.1: A disaggregated deployment as a queueing model. A turn waits for KV memory at a prefill instance (it is admitted only when its whole context fits, in arrival order), is prefilled one request at a time in arrival order, sends its KV cache over a link whose bandwidth concurrent transfers share, waits for KV memory at a decode instance, and decodes in a batch that shares the instance’s throughput. Sessions loop through a tool call back to the prefill instance, where the prefix cache decides whether the next turn is a hit (short prefill work) or a miss (the whole context is recomputed).

delay stations, closed loops) are defined and analysed in later lectures; here the serving system itself is defined, first in the picture and then exactly, so that every later theorem has a place in it.

The figure is a picture, not a definition: it does not say when a waiting request is admitted, what happens to its KV memory afterwards, or who decides a hit. The rest of this section makes it a definition. We first define a small language in which a serving deployment is a program, and give it a precise meaning (Section 1.2.1); the language knows nothing about tokens or prefill costs. We then write the deployment of Figure 1.1 as one program, with its workload and costs as the program’s parameters (Section 1.2.2), and read off what that program implies (Section 1.2.3). The tools of later lectures are needed only to compute averages of the quantities defined here.

1.2.1 A language for serving deployments

Requests and sessions. A *request* is one turn of an agent: it arrives at a prefill instance, waits, is prefilled, transferred and decoded, and leaves. A *session* is the agent program that sends these requests: it returns after each tool call and leaves only when the program ends. New sessions arrive from outside as a Poisson process, but the requests of a live session do not arrive independently of the system: the next one exists only after the previous one has been decoded and its tool call has returned. In the language the object that moves is therefore the session, and each request is one pass of the session through a loop.

A serving deployment does four things to a request: it makes it wait until its KV cache fits in a memory pool, runs it on a stage (a prefill instance, a link, a decode instance, a tool), frees its memory (possibly keeping the prefix cached), and sends it somewhere next. We give each of these an instruction. The resulting language, which we call ROUTE, describes a system by a *deployment* (its memory pools and stages), a *workload* (how sessions arrive and how their turns evolve), and a *route*, the path a session follows through the deployment.

Definition 1.7 (Syntax of ROUTE)

A ROUTE program is a triple $(\mathcal{D}, \mathcal{W}, \pi_0)$ of a deployment, a workload and a route.

$$\begin{aligned}
 \text{deployment item } d &::= \mathbf{pool} \ m \ (M, E) \mid \mathbf{stage} \ s : k \\
 \text{stage kind } k &::= \text{serial} \mid \text{shared}(\varphi) \mid \text{external} \mid \text{step}(B, \tau) \\
 \text{workload } \mathcal{W} &::= \mathbf{arrivals} \ (\Lambda, y_0, G) \\
 \text{action } \alpha &::= \mathbf{turn} \mid \mathbf{admit} \ m \ c \mid \mathbf{free} \ m \mid \mathbf{free} \ m \ \mathbf{cache} \ \ell \\
 &\quad \mid \mathbf{run} \ s \ w \mid \mathbf{run} \ s \ (w_p, w_d) \ \mathbf{growing} \ m \\
 \text{route } \pi &::= \epsilon \mid \alpha; \pi \mid \mathbf{loop} \ \pi \mid \mathbf{branch}_p \ \pi_1 \ \pi_2 \mid \mathbf{end}
 \end{aligned}$$

Here m names a KV memory pool of M bytes with eviction order E , s names a stage, and $p \in [0, 1]$. A *serial* stage runs one request at a time, in arrival order, at rate 1 (a prefill instance); a *shared*(φ) stage runs all its requests at once and splits a throughput $\varphi(n)$ equally among the n present (a decode batch, a link); an *external* stage runs every request on its own, with no waiting (a tool call); a *step*(B, τ) stage is an engine that runs iterations, handing a budget of B tokens per iteration to its jobs in their order of entry, one token to a job in its decode phase and as many as its remaining prefill work allows to a job in its prefill phase, and taking $\tau(\cdot)$ seconds for an iteration of the tokens it scheduled (a colocated replica with chunked prefill). The workload has a session arrival rate Λ , a measurable **attribute space** $\mathcal{X}$ with the attributes $y_0 \in \mathcal{X}$ of a new session, and a **turn kernel** G , a transition kernel on $\mathcal{X}$: $G(y, \cdot)$ is the law of a session's next-turn attributes given its current ones. A session's attributes are $x = (y, H) \in \mathcal{X} \times \{0, 1\}$, where y is set by the workload and the **hit indicator** H by the deployment. The arguments c (bytes to allocate), ℓ (bytes to keep cached, $\ell \leq c$) and w (work) are nonnegative measurable functions of x . Sequencing is associative, with ϵ as unit.

We write **hold** $m \ c \ \{\pi\}$ **cache** ℓ for **admit** $m \ c$; π ; **free** $m \ \mathbf{cache} \ \ell$: every program we write frees what it admits, and the scoped form makes that balance syntactic. Informally: **turn** draws the session's next-turn attributes from G ; **admit** $m \ c$ joins pool m 's admission queue and leaves it with c bytes allocated; **free** $m \ \mathbf{cache} \ \ell$ frees the allocation but keeps ℓ bytes of it as a cached prefix, which the pool may evict later; **run** $s \ w$ joins stage s with w units of work and continues when the work is done; **run** $s \ (w_p, w_d) \ \mathbf{growing} \ m$ joins a step stage with w_p prefill and w_d decode tokens, and before each iteration enlarges its allocation in pool m to the tokens it will have computed; **branch** _{p} continues with π_1 with probability p and with π_2 otherwise; **end** ends the session.

Semantics. The meaning of a program is a stochastic process whose state is the following configuration. For a finite sequence q we write $r \cdot q$ for q with r in front, $q \cdot r$ for q with r appended, $q - r$ for q with r removed, and for a partial map f into $\mathbb{R}_+$ (or into pairs whose first entry is in $\mathbb{R}_+$) we write $|f|$ for the sum of its (first) values.

Definition 1.8 (Configuration)

A **configuration** Γ of a program $(\mathcal{D}, \mathcal{W}, \pi_0)$ consists of

- (a) the time $t \geq 0$ and the finite set $\mathcal{R}$ of live sessions;
- (b) for each session $r \in \mathcal{R}$: its attributes $x_r = (y_r, H_r)$, its continuation π_r (the rest of its route), its status $\text{st}_r \in \{\text{ready}\} \cup \{\text{queued}(m)\} \cup \{\text{in}(s)\}$, and its remaining work $\omega_r \geq 0$;
- (c) for each pool m : its admission queue q_m , a finite sequence of sessions; its allocation h_m , a partial map from sessions to bytes; and its prefix cache C_m , a partial map from sessions to pairs (bytes, time of last use);
- (d) for each stage s : the sequence Q_s of sessions at the stage, in order of entry.

Every reachable configuration satisfies the memory invariant

$$|h_m| + |C_m| \leq M \quad \text{for every pool } m \text{ of } M \text{ bytes.} \quad (1.1)$$

The program runs by two kinds of steps. *Commands* take no time and change the configuration when their guard holds. *Flow* lets time pass while no command is enabled. For a pool with eviction order E , $\text{evict}_E(C, b)$ deletes entries of C in the order E until $|C| \leq b$; for LRU the order is increasing time of last use. We write “ r at $\alpha; \pi'$ ” for $\text{st}_r = \text{ready}$ and $\pi_r = \alpha; \pi'$.

Definition 1.9 (Operational semantics of ROUTE)

The commands are the following guarded assignments, written $[\text{NAME}] \text{ guard} \longrightarrow \text{effect}$.

[TURN]	r at turn ; π'	$\longrightarrow \text{draw } y' \sim G(y_r, \cdot); y_r := y'; \pi_r := \pi'$
[QUEUE]	r at admit m c ; π'	$\longrightarrow q_m := q_m \cdot r; \text{st}_r := \text{queued}(m); \pi_r := \pi'$
[ADMIT]	$q_m = r \cdot q', h_m + c(x_r) \leq M$	$\longrightarrow H_r := \mathbf{1}\{r \in \text{dom } C_m\}; C_m := C_m - r;$ $C_m := \text{evict}_E(C_m, M - h_m - c(x_r));$ $h_m(r) := c(x_r); q_m := q'; \text{st}_r := \text{ready}$
[FREE]	r at free m [cache ℓ]; π'	$\longrightarrow h_m := h_m - r; [C_m(r) := (\ell(x_r), t)];$ $\pi_r := \pi'$
[RUN]	r at run s w ; π'	$\longrightarrow \omega_r := w(x_r); Q_s := Q_s \cdot r;$ $\text{st}_r := \text{in}(s); \pi_r := \pi'$
[FINISH]	$\text{st}_r = \text{in}(s), \omega_r = 0$	$\longrightarrow Q_s := Q_s - r; \text{st}_r := \text{ready}$
[BRANCH]	r at branch _{p} $\pi_1 \pi_2; \pi'$	$\longrightarrow \pi_r := \pi_1; \pi' \text{ w.p. } p, \text{ else } \pi_r := \pi_2; \pi'$
[LOOP]	r at loop $\pi; \pi'$	$\longrightarrow \pi_r := \pi; \text{loop } \pi; \pi'$
[END]	r at end ; π'	$\longrightarrow \mathcal{R} := \mathcal{R} - r$

The brackets in FREE are present exactly when the action has **cache**. All random draws are independent of each other and of the past. Commands are *urgent*: while any command is enabled, one is executed (the enabled command of the session that arrived first). When none is enabled, FLOW lets time advance: each session at a stage works off its remaining work at rate

$$v_r = \begin{cases} \mathbf{1}\{r \text{ is the first entry of } Q_s\} & s : \text{serial}, \\ \varphi(|Q_s|)/|Q_s| & s : \text{shared}(\varphi), \\ 1 & s : \text{external}, \end{cases} \quad \frac{d\omega_r}{dt} = -v_r, \quad (1.2)$$

until the first time some ω_r reaches 0 (which enables FINISH) or a session arrives. At each epoch of the Poisson process of rate Λ , the command ARRIVE adds a new session r to $\mathcal{R}$ with $y_r = y_0, H_r = 0, \pi_r = \pi_0$ and $\text{st}_r = \text{ready}$.

A step stage does not flow continuously: it advances by iterations. At the start of an iteration its jobs are taken in order of entry and each is handed $\min(\text{remaining}, \text{budget left})$ tokens (one for a decode phase); a **growing** job first enlarges its allocation, and if the pool has no room the most recently admitted holder is *preempted*: its job leaves the stage, its allocation is freed with the prefix it computed cached, and it rejoins the head of the pool's queue to be admitted again. The iteration ends τ seconds later and its tokens are then applied.

ADMIT is where the admission rule lives. Only the first session in the queue can be admitted, so a request that does not fit blocks every request behind it; the guard counts only the bytes allocated to admitted requests, not the cache, so a cached prefix never blocks an admission; instead, the eviction in the second line of ADMIT deletes as many cached prefixes as the new allocation needs, which keeps (1.1). The hit indicator is decided at the same moment, by whether the session's own prefix survived until its admission. FREE also keeps the invariant, since it frees c bytes and caches $\ell \leq c$ of them.

Algorithm 1 Interpreter of a ROUTE program up to time $t_{\max}$

Require: deployment $\mathcal{D}$, workload $\mathcal{W}$, route π_0 , horizon $t_{\max}$

```
1:  $t \leftarrow 0$ ;  $\mathcal{R} \leftarrow \emptyset$ ; all queues, allocations, caches and stages empty
2:  $a \leftarrow$  a draw from  $\text{Exp}(\Lambda)$  ▷ next session arrival
3: while  $t < t_{\max}$  do
4:   while some command is enabled do ▷ zero-time phase
5:     execute the enabled command of the earliest-arrived session
6:   compute  $v_r$  for every session at a stage by (1.2)
7:    $e \leftarrow$  the earliest end of a running iteration of a step stage ( $\infty$  if none)
8:    $\Delta \leftarrow \min(a - t, e - t, \min_{r: v_r > 0} \omega_r / v_r)$ 
9:   for every session  $r$  at a stage do
10:     $\omega_r \leftarrow \omega_r - v_r \Delta$ 
11:    $t \leftarrow t + \Delta$ 
12:   if  $t = a$  then
13:     ARRIVE;  $a \leftarrow t +$  a draw from  $\text{Exp}(\Lambda)$ 
14:   if  $t = e$  then ▷ apply the iteration's tokens; the next one is assigned
15:     end the iteration; start the next one after the zero-time phase
```

Production schedulers. Three refinements of ADMIT are needed to write a production engine such as vLLM as a program, and none changes the invariant. (i) The queue of a pool may be served by a step stage: admission happens only at the start of an iteration, with the budget the running jobs leave, so a waiting request's cached prefix is not protected until then and can be evicted while it waits. (ii) The guard may ask for more than is allocated: vLLM admits a request only if its whole prompt fits, but allocates only the blocks of its first chunk and grows from there. (iii) A request may reuse only part of its cached prefix (the blocks it shares with its previous turn); the rest stays cached, unusable, until it is evicted. With these rules the program of the vLLM v1 engine reproduces the real scheduler step for step, and it keeps the cached prefixes of finished sessions, as END does: the cache does not know that a session has left.

The semantics is well defined for programs in which every **loop** body contains a **run** with positive work (a step stage's jobs are not in (1.2): they advance only at the end of an iteration): then between two flow steps each session executes only finitely many commands. It is also an algorithm. Algorithm 1 runs it, and a discrete-event simulator of a serving system is an implementation of this loop.

1.2.2 The disaggregated replica

A program needs a workload and the functions c , ℓ , w . For the deployment of Figure 1.1 they come from the agentic workload and from the costs of one replica.

Definition 1.10 (Agentic workload)

Sessions arrive at the epochs $A_1 < A_2 < \dots$ of a Poisson process of rate Λ (Definition 1.17). The attributes of a turn are $y = (K, n, o, T, Z) \in \mathcal{X} = \mathbb{R}_+^5$.

- (a) *Tokens*. Turn j of session i carries a prefix of K_{ij} tokens that the prefill instance has already processed, n_{ij} new tokens, and produces o_{ij} output tokens. Its context is $T_{ij} = K_{ij} + n_{ij}$, and

$$K_{i1} = 0, \quad K_{i,j+1} = T_{ij}, \quad n_{i,j+1} \geq o_{ij}. \quad (1.3)$$

The new tokens of the next turn contain the previous output, which was produced on the decode instance, and the tool's output.

- (b) *Think times*. After turn j the session calls a tool for a time $Z_{ij} \geq 0$ with mean Z .
(c) *Turn kernel*. A new session has $y_0 = 0$. Given the current turn y , $G(y, \cdot)$ draws (n', o', Z') with $n' \geq o$ and sets $K' = T$ and $T' = K' + n'$, so that successive turns satisfy (1.3).

Definition 1.11 (Costs)

A replica has prefill constants $a, b > 0$, a KV size of κ bytes per token, a link of bandwidth B with fixed overhead x_0 , a decode capacity function φ , and memory pools of M_P and M_D bytes. For turn (i, j) :

- (a) Prefilling n new tokens on a cached prefix of K tokens takes

$$P(n, K) = a n + b n (K + n/2) \quad (1.4)$$

seconds. With the hit indicator H_{ij} (whether the prefix is still cached when the turn is admitted), the prefill work is

$$S_{ij} = H_{ij} P(n_{ij}, K_{ij}) + (1 - H_{ij}) P(T_{ij}, 0) = P(n_{ij}, K_{ij}) + (1 - H_{ij}) \Delta S(K_{ij}), \quad (1.5)$$

where $\Delta S(K) = aK + bK^2/2$ is the extra work of a miss.

- (b) The transfer of the context to the decode instance is the work $X_{ij} = x_0 + \kappa T_{ij}/B$ seconds.
(c) The decode work is $D_{ij} = o_{ij}$ tokens. A batch of m running turns takes $\tau(m)$ seconds per step and advances each turn by one token, so its capacity is $\varphi(m) = m/\tau(m)$ tokens per second.
(d) The memory footprint is $c_{ij} = \kappa T_{ij}$ bytes on either instance.

Example 1.12 (The disaggregated replica)

The deployment of Figure 1.1 is the program

pool mem_P (M_P , LRU)	stage prefill : serial
pool mem_D (M_D , LRU)	stage link : shared(1)
arrivals (Λ , y_0 , G)	stage decode : shared(φ)
	stage tool : external

$\pi_0 = \text{turn}; \text{loop} ($	
admit mem_P c ;	<i>wait for KV memory at the prefill instance</i>
run prefill S ;	<i>the first token is out when this ends</i>
run link X ;	<i>KV transfer, bandwidth shared</i>
free mem_P cache κT ;	<i>memory freed, prefix stays cached</i>
admit mem_D c ;	<i>wait for KV memory at the decode instance</i>
run decode D ;	<i>batched decode</i>
free mem_D ;	
branch _{p} (run tool Z ; turn) (end))	<i>tool call and next turn, or end</i>

with the workload (Λ, y_0, G) of Definition 1.10 and c , S , X , D from Definition 1.11. Because **branch** _{p} ends the loop with probability $1 - p$ after each turn, session i has J_i turns with $\mathbb{P}(J_i \geq j + 1 \mid J_i \geq j) = p$, so $\mathbb{E}[J_i] = 1/(1 - p)$. Other deployments are other programs: a replica without a prefix cache drops **cache** κT , and every turn is then a miss (Exercise 1.3); a colocated replica is a shorter program with one pool and no link (Exercise 1.4).

1.2.3 What the program implies

Running the program of Example 1.12 produces, for every request, seven epochs. Number requests $k = 1, 2, \dots$ in the order of their QUEUE at mem_P . We write $\sigma(k)$ for the session of request k , and S_k , X_k , D_k , c_k , H_k and Z_k for its quantities. Let

$$a_k \leq \alpha_k \leq s_k \leq f_k \leq e_k \leq \alpha'_k \leq d_k$$

be the times of its QUEUE at mem_P , its ADMIT there, the moment it becomes the first entry of Q_{prefill} , its FINISH at **prefill**, its FINISH at **link**, its ADMIT at mem_D , and its FINISH at **decode**. Let $R_P(t) = \{k : \alpha_k \leq t < e_k\}$ be the requests holding memory at the prefill instance.

Lemma 1.13 (Sample-path equations of the replica)

Under Definition 1.9, the program of Example 1.12 satisfies

$$\alpha_k = \inf \left\{ t \geq \max(a_k, \alpha_{k-1}) : \sum_{l \in R_P(t)} c_l + c_k \leq M_P \right\}, \quad (1.6)$$

$$s_k = \max(\alpha_k, f_{k-1}), \quad f_k = s_k + S_k, \quad (1.7)$$

$$a_{k^+} = d_k + Z_k \quad \text{if the session continues,} \quad (1.8)$$

where k^+ is the next request of session $\sigma(k)$, and $H_k = \mathbf{1}\{\sigma(k)\text{'s prefix is in } C_{\text{mem}_P} \text{ at } \alpha_k\}$. The link and the decode batch drain X_k and D_k at the rates (1.2).

Proof. A request joins the admission queue of mem_P at a_k and can be admitted only at its head, after α_{k-1} , and only once the guard of ADMIT holds, which is (1.6). After its admission it executes RUN at once, so it enters Q_{prefill} in admission order, which is arrival order; by (1.2) it is worked at rate 1 exactly when it is first, that is from the FINISH of its predecessor or its own entry,

whichever is later, until S_k is worked off, which is (1.7). After FINISH at decode the session frees its memory, branches, runs Z_k at the external stage (rate 1, no waiting), executes TURN and LOOP, and queues again, all in zero time except the tool, which is (1.8). The statement about H_k is the first assignment of ADMIT. $\square$

Eq. (1.8) is the closed loop: request arrival times depend on completion times, which depend on the congestion. The request rate is nevertheless fixed by the session rate. If the system is stable, every session leaves after its J_i turns (Example 1.12), and

$$\lambda = \Lambda \mathbb{E}[J] = \frac{\Lambda}{1-p}. \quad (1.9)$$

The stages as queues. Each stage kind is a queue studied in one later lecture. A serial stage is a single first-in first-out server, and (1.7) is its recursion: the M/G/1 queue of Lecture 3 when arrivals are Poisson and the closed system of Lecture 4 when a finite number of sessions circulate. A shared stage is a processor-sharing station (Lecture 2), and an external stage is a delay station (Lecture 4). A pool is a queue too. If every context needs about c bytes, the guard of ADMIT reads $|R_P(t)| < m_P$ with $m_P = \lfloor M_P/c \rfloor$ slots: a multi-server queue $\cdot/G/m_P$ (Lecture 2) whose holding time is the time from ADMIT to FREE,

$$e_k - \alpha_k = \underbrace{(s_k - \alpha_k)}_{\text{wait for prefill}} + S_k + \underbrace{(e_k - f_k)}_{\text{transfer}},$$

and likewise $m_D = \lfloor M_D/c \rfloor$ slots held for $d_k - \alpha'_k$ at the decode instance. With unequal contexts a request needs several slots, and the queue has no closed form.

The cache chooses the prefill work. In classical queueing theory the service-time law is given. Here, by (1.5) and ADMIT, the cache chooses it: the eviction order E decides H_k , and

$$\mathbb{E}[S] = \mathbb{E}[P(n, K)] + \mathbb{E}[(1-H) \Delta S(K)]. \quad (1.10)$$

Only the second term depends on the policy, and it depends on it through which sessions miss, weighted by $\Delta S(K)$, which grows as K^2 . The central question of Lecture 3 is how a performance measure changes when the distribution of S changes; (1.10) says the eviction policy is what changes it.

Performance. The first token is produced when the prefill ends, so

$$\text{TTFT}_k = f_k - a_k = \underbrace{(\alpha_k - a_k)}_{W_{P,k}^M} + \underbrace{(s_k - \alpha_k)}_{W_{P,k}} + S_k, \quad (1.11)$$

the wait for KV memory at the prefill instance, the wait for the prefill instance, and the prefill work. The transfer and the decode come after the first token and do not enter it; they set when the rest of the answer arrives, and they reach the TTFT only through memory, because a request keeps its prefill allocation until e_k . The objective of the scheduler is the time average of the number of requests in the system,

$$L = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t N(u) du, \quad N(u) = \#\{k : a_k \leq u < d_k\},$$

which by Little's law (Theorem 1.24) equals λW with W the average of $d_k - a_k$ over requests and λ from (1.9). No stage can be stable unless its load is below its capacity:

$$\rho_P = \lambda \mathbb{E}[S] < 1, \quad \rho_L = \lambda \mathbb{E}[X] < 1, \quad \lambda \mathbb{E}[D] < \max_{m \leq m_D} \varphi(m). \quad (1.12)$$

By (1.10), the first condition already depends on the eviction policy.

What the model leaves out. Work is a continuous quantity rather than whole tokens and memory blocks; an instance's scheduler steps are replaced by the continuous flow (1.2); the allocation c_k ignores the o_k tokens a decode adds, and a running decode that needs a block when none is free is preempted and later recomputed, which the step stage with **growing** expresses but the replica of Example 1.12 does not use; and by (1.8) request arrivals are not Poisson. Lecture 5 returns to these gaps once the tools are in place.

1.3 The exponential distribution and memorylessness

Definition 1.14 (Exponential distribution)

A random variable $X \geq 0$ with $\mathbb{P}(X > t) = e^{-\mu t}$ for $t \geq 0$ has the **exponential distribution** with **rate** $\mu > 0$, written $X \sim \text{Exp}(\mu)$. Then $\mathbb{E}[X] = 1/\mu$ and $\mathbb{E}[X^2] = 2/\mu^2$.

Proposition 1.15 (Memorylessness [20])

If $X \sim \text{Exp}(\mu)$, then for all $s, t \geq 0$

$$\mathbb{P}(X > s + t \mid X > s) = \mathbb{P}(X > t). \quad (1.13)$$

Conversely, if a random variable with $0 < X < \infty$ a.s. satisfies (1.13), then $X \sim \text{Exp}(\mu)$ for some $\mu \in (0, \infty)$.

Proof. The forward direction is $e^{-\mu(s+t)}/e^{-\mu s} = e^{-\mu t}$. Converse: let $G(t) = \mathbb{P}(X > t)$. Then (1.13) reads $G(s+t) = G(s)G(t)$. G is nonincreasing and right-continuous, and $G(0) = \mathbb{P}(X > 0) = 1$. By induction $G(nt) = G(t)^n$, hence $G(1) = G(1/n)^n$ and $G(m/n) = G(1/n)^m$. If $G(1) = 0$, then $G(1/n) = 0$ for every n , which contradicts $G(1/n) \rightarrow G(0) = 1$ by right-continuity. If $G(1) = 1$, then $G(r) = 1$ for every rational r , and by monotonicity $G \equiv 1$, that is $X = \infty$ a.s., contradicting the assumption. Hence $G(1) = e^{-\mu}$ for some $\mu \in (0, \infty)$, and $G(r) = e^{-\mu r}$ on the rationals. For arbitrary t , take rationals $r \downarrow t$; right-continuity gives $G(t) = e^{-\mu t}$. $\square$

Memorylessness says that the remaining service time of a customer who has already received s units of service has the same distribution as the service time of a new customer. This is why, in a system with exponential service, the number in system $N(t)$ alone determines the future, and $N(t)$ is a continuous-time Markov chain (Lecture 2). The following fact is also used often.

Lemma 1.16 (Minimum of exponentials [20])

If $X_i \sim \text{Exp}(\mu_i)$, $i = 1, \dots, n$, are independent, then $\min_i X_i \sim \text{Exp}(\sum_i \mu_i)$.

Proof. $\mathbb{P}(\min_i X_i > t) = \prod_i \mathbb{P}(X_i > t) = e^{-(\sum_i \mu_i)t}$. $\square$

1.4 The Poisson process

Definition 1.17 (Counting process and Poisson process)

A **counting process** $A = (A(t))_{t \geq 0}$ is a nondecreasing, right-continuous, $\mathbb{N}$ -valued process with $A(0) = 0$ and jumps of size one only. A counting process A is a **Poisson process** with **rate** $\lambda > 0$ if

1. (**independent increments**) for disjoint intervals $(s_1, t_1], \dots, (s_n, t_n]$ the increments $A(t_j) - A(s_j)$ are independent;
2. (**stationary Poisson increments**) for all $s \geq 0, t > 0$, $A(s+t) - A(s) \sim \text{Poisson}(\lambda t)$, that is $\mathbb{P}(A(s+t) - A(s) = n) = e^{-\lambda t} (\lambda t)^n / n!$.

$S_n = \inf\{t : A(t) \geq n\}$ is the n -th arrival time, and $T_n = S_n - S_{n-1}$ ($S_0 = 0$) is the n -th **interarrival time**.

Theorem 1.18 (Exponential interarrival times [20, §2.2])

A is a Poisson process with rate λ if and only if the interarrival times $T_1, T_2, \dots$ are independent and each is $\text{Exp}(\lambda)$.

Proof ($\Rightarrow$). For $0 = t_0 < t_1 < \dots < t_n$ and small $h > 0$ consider the event $E_h = \{S_j \in (t_j, t_j + h], j = 1, \dots, n\}$. Let F_h be the event that there is no arrival in $(t_{j-1} + h, t_j]$ and exactly one in each $(t_j, t_j + h]$. Then $F_h \subset E_h$, and $E_h \setminus F_h$ is contained in the event that some interval of length h has two or more arrivals, which has probability $O(h^{n+1})$. By independent and Poisson increments,

$$\mathbb{P}(F_h) = e^{-\lambda(t_1 + (t_2 - t_1 - h) + \dots + (t_n - t_{n-1} - h))} (\lambda h e^{-\lambda h})^n = \lambda^n h^n e^{-\lambda t_n} (1 + O(h)).$$

Hence $(S_1, \dots, S_n)$ has density $\lambda^n e^{-\lambda s_n}$ on $\{0 < s_1 < \dots < s_n\}$. The linear map $T_j = S_j - S_{j-1}$ has Jacobian one and $s_n = \sum_j T_j$, so $(T_1, \dots, T_n)$ has density $\prod_{j=1}^n \lambda e^{-\lambda t_j}$, that is, independent $\text{Exp}(\lambda)$. The converse is a standard argument using that S_n is Gamma distributed together with memorylessness, and we omit it [20, §2.2]. $\square$

Proposition 1.19 (Superposition and thinning [19])

- (i) (**Superposition**) If A_1, A_2 are independent Poisson processes with rates λ_1, λ_2 , then $A_1 + A_2$ is a Poisson process with rate $\lambda_1 + \lambda_2$.
- (ii) (**Thinning**) If each point of a Poisson process with rate λ is kept with probability p and discarded with probability $1 - p$, independently of everything else, then the kept points and the discarded points are Poisson processes with rates $p\lambda$ and $(1 - p)\lambda$, and they are independent.

Proof. (i) Independent increments follow from the independent increments of the two processes and their mutual independence. For the increment over one interval, the generating function of independent Poisson variables is $\mathbb{E}[z^{X_1 + X_2}] = e^{\lambda_1 t(z-1)} e^{\lambda_2 t(z-1)} = e^{(\lambda_1 + \lambda_2)t(z-1)}$, so the sum is $\text{Poisson}((\lambda_1 + \lambda_2)t)$.

(ii) Let X be the original increment over an interval of length t , X_1 the number kept, and $X_2 = X - X_1$ the number discarded. Given $X = j + k$, the conditional probability that $X_1 = j$ is binomial, $\binom{j+k}{j} p^j (1-p)^k$, so

$$\mathbb{P}(X_1 = j, X_2 = k) = e^{-\lambda t} \frac{(\lambda t)^{j+k}}{(j+k)!} \binom{j+k}{j} p^j (1-p)^k = e^{-p\lambda t} \frac{(p\lambda t)^j}{j!} \cdot e^{-(1-p)\lambda t} \frac{((1-p)\lambda t)^k}{k!}.$$

Thus on one interval X_1, X_2 are independent Poisson variables. On disjoint intervals the original increments and the coin flips are all independent, so all increments of the two processes are independent. $\square$

In LLM serving

If sessions arrive as a Poisson process of rate Λ and each turn is routed independently to one of J prefill instances, then by Proposition 1.19(ii) each instance also sees Poisson arrivals. The turns within a session, however, go around a closed loop (prefill $\rightarrow$ link $\rightarrow$ decode $\rightarrow$ tool), so the turn arrivals are not exactly Poisson. The closed models of Lecture 4 are the tool for this gap.

1.5 PASTA: Poisson arrivals see time averages

The distribution of the state seen by arriving customers generally differs from the time-average distribution (Example 1.21(b) below gives an example). With Poisson arrivals this discrepancy disappears.

Definition 1.20 (Lack of anticipation)

Given a state process $X = (X(t))_{t \geq 0}$ and an arrival process A , we say that A has **lack of anticipation** with respect to X if for every t the future increments $\{A(t+s) - A(t) : s \geq 0\}$ are independent of the past $\{X(u) : u \leq t\} \cup \{A(u) : u \leq t\}$.

In a queueing system this assumption says that future arrivals are independent of the system state so far. Note the direction: the state may (and does) depend on past arrivals; what is excluded is that future arrivals react to the state. Poisson arrivals from outside satisfy it naturally. Arrivals in a closed system (Lecture 4) do not.

Example 1.21 (One process with lack of anticipation, one without)

(a) *Lack of anticipation: the M/M/1 queue.* Customers come from a large outside population as a Poisson process, independently of the service times. The queue length $X(t)$ is a function of the arrivals before t and the service times, and by independent increments the arrivals after t are independent of both. An arriving customer is like a raindrop falling on the queue: the rain does not know how long the queue is. So the arrival instants are “random inspection times”, and by Theorem 1.23 the fraction of arrivals that find the server busy equals the fraction of time it is busy, ρ .

(b) *No lack of anticipation: a single agent session.* One session alternates between a tool call (mean Z) and a turn at a server (mean $1/\mu$). Its next turn can arrive only after its previous turn has left the server. If $X(t) = 1$ (the turn is being served), then no arrival can happen until the service ends; if $X(t) = 0$, an arrival may come at any moment. Future arrivals depend on the current state, so the condition fails. The consequence is visible: every arrival finds the server idle, while the server is busy a fraction $(1/\mu)/(Z + 1/\mu) > 0$ of the time. Arrivals do not see time averages. With N sessions the same effect is weaker but present, and Lecture 4 replaces PASTA by the arrival theorem for exactly this reason.

Remark 1.22 (Why study PASTA if sessions violate it?)

Example 1.21(b) shows that a single agent session does not have lack of anticipation. PASTA is still the right starting point, for three reasons.

1. *The prefill model rests on it.* The prefill instance is modelled as an M/G/1 queue, and the proof of the Pollaczek–Khinchine formula (Theorem 3.4) uses PASTA. Session feedback makes this an approximation, to be tested against the system. One cannot judge an approximation without knowing what it approximates.
2. *Many sessions look Poisson.* Each session reacts to the state, but the superposition of many independent sessions, each contributing a small share of the arrivals, is close to a Poisson process; this is the Palm–Khinchine theorem [9]. New sessions themselves arrive as a Poisson process in the model, so the M/G/∞ and Campbell calculations of Lecture 4 use Poisson arrivals without approximation.
3. *The exact closed-system answer is a corrected PASTA.* The arrival theorem (Theorem 4.3) says that an arriving turn sees the time averages of the system with one session fewer, and it reduces to PASTA as the number of sessions grows (Remark 4.4). For exponential work, Proposition 4.6(ii) shows that the open formula built on PASTA overstates the wait at equal utilisation, so PASTA gives a conservative estimate whose error the finite-population model quantifies.

Theorem 1.23 (PASTA [23])

Let A be a Poisson process with rate λ , X a state process with left limits, and B a set of states, and suppose A has lack of anticipation with respect to X . If $U(t) = \mathbf{1}\{X(t^-) \in B\}$ is left-continuous, then for every $T > 0$

$$\mathbb{E}\left[\sum_{k: S_k \leq T} \mathbf{1}\{X(S_k^-) \in B\}\right] = \lambda \mathbb{E}\left[\int_0^T \mathbf{1}\{X(t) \in B\} dt\right]. \quad (1.14)$$

Consequently, if the limits of the two averages $\frac{1}{T} \int_0^T \mathbf{1}\{X(t) \in B\} dt$ and $\frac{1}{A(T)} \sum_{k \leq A(T)} \mathbf{1}\{X(S_k^-) \in B\}$ exist with probability one, they are equal: the fraction of arrivals that find the system in B equals the fraction of time it spends in B .

Proof. The left side is $\int_0^T U(t) dA(t)$. For the partition $t_j = jT/n$ of $[0, T]$, define the step function $U_n(t) = \sum_j U(t_j) \mathbf{1}\{t \in (t_j, t_{j+1}]\}$. $U(t_j)$ is determined by the past up to time t_j , and by lack of anticipation the increment $A(t_{j+1}) - A(t_j)$ is independent of that past, so

$$\mathbb{E}\left[\int_0^T U_n dA\right] = \sum_j \mathbb{E}[U(t_j)] \mathbb{E}[A(t_{j+1}) - A(t_j)] = \lambda \sum_j \mathbb{E}[U(t_j)] (t_{j+1} - t_j) = \lambda \mathbb{E}\left[\int_0^T U_n(t) dt\right].$$

Since U is left-continuous, $U_n(t) \rightarrow U(t)$ for every t . The integrands on the two sides are bounded by $A(T)$ and T respectively, and $\mathbb{E}[A(T)] = \lambda T < \infty$, so by dominated convergence $\mathbb{E}[\int_0^T U dA] = \lambda \mathbb{E}[\int_0^T U dt]$ as $n \rightarrow \infty$. Finally, the set of t with $X(t^-) \neq X(t)$ is countable, so replacing $U(t)$ by $\mathbf{1}\{X(t) \in B\}$ on the right does not change the integral. The statement about limits follows from a sample-path version of (1.14) (a law of large numbers applied to a martingale), for which we cite [23]. $\square$

In Lecture 3, PASTA becomes a key step of the Pollaczek–Khinchine formula in the form “the mean work an arriving turn finds waiting equals the time-average work waiting”.

1.6 Little's law

Theorem 1.24 (Little's law [11, 22])

For a sequence of customers (a_k, d_k) with $a_k \leq d_k < \infty$ and a_k nondecreasing, suppose the two limits

$$\lambda = \lim_{t \rightarrow \infty} \frac{A(t)}{t} \in (0, \infty), \quad W = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n W_k < \infty$$

exist. Then $L = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t N(s) ds$ exists and

$$L = \lambda W. \quad (1.15)$$

Proof. Since $N(s) = \sum_k \mathbf{1}\{a_k \leq s < d_k\}$, we have $\int_0^t N(s) ds = \sum_k |[a_k, d_k) \cap [0, t]|$. Each term is at most W_k , is zero if $a_k > t$, and equals W_k exactly if $d_k \leq t$. Therefore

$$\sum_{k: d_k \leq t} W_k \leq \int_0^t N(s) ds \leq \sum_{k: a_k \leq t} W_k = \sum_{k=1}^{A(t)} W_k. \quad (1.16)$$

Upper bound: since $A(t) \rightarrow \infty$, $\frac{1}{t} \sum_{k \leq A(t)} W_k = \frac{A(t)}{t} \cdot \frac{1}{A(t)} \sum_{k \leq A(t)} W_k \rightarrow \lambda W$.

Lower bound: with $\bar{W}_n = \frac{1}{n} \sum_{k \leq n} W_k$ we have $\frac{W_n}{n} = \bar{W}_n - \frac{n-1}{n} \bar{W}_{n-1} \rightarrow W - W = 0$. Also $A(a_n) \geq n$ and $A(a_n^-) \leq n-1$, so $a_n/n \rightarrow 1/\lambda$, and therefore $W_n/a_n \rightarrow 0$. For any $\varepsilon > 0$ there is k_0 such that $d_k = a_k + W_k \leq (1+\varepsilon)a_k$ for $k \geq k_0$. Then for $t \geq \max_{k < k_0} d_k$, every customer with $a_k \leq t/(1+\varepsilon)$ has $d_k \leq t$, so

$$\frac{1}{t} \int_0^t N(s) ds \geq \frac{1}{t} \sum_{k=1}^{A(t/(1+\varepsilon))} W_k \rightarrow \frac{\lambda W}{1+\varepsilon}.$$

Letting $\varepsilon \downarrow 0$, the liminf is at least λW . Together with the upper bound this gives (1.15). $\square$

The proof uses no distributional assumption and no service discipline. It is a deterministic theorem that holds on every sample path on which the limits exist. So depending on how we draw the boundary of “the system”, we obtain several identities.

Corollary 1.25 (Utilisation law [12])

Under the assumptions of Theorem 1.24, consider a single-server system and suppose the average of the service times $\frac{1}{n} \sum_{k \leq n} S_k$ converges to $\mathbb{E}[S]$.

- (i) The fraction of time the server is busy is $\rho = \lambda \mathbb{E}[S]$. In particular $\rho \leq 1$ in a stable system.
- (ii) For the waiting room, $L_q = \lambda W_q$, and under FIFO $L = L_q + \rho$.

Proof. (i) Take “the server” alone as the system. Customer k enters it when its service starts and leaves S_k later. Every customer is eventually served, so the arrival rate of this subsystem is also λ , and the customer average of its sojourn times is $\mathbb{E}[S]$. The number in this subsystem is 1 when the server is busy and 0 otherwise, so its time average is the fraction of time the server is busy. By Theorem 1.24 this equals $\lambda \mathbb{E}[S] = \rho$, and being a fraction it is at most one. (ii) Taking the waiting room alone as the system, the sojourn times are $W_{q,k}$, so $L_q = \lambda W_q$. Taking time averages in $N = N_q + (\text{number in service})$ gives $L = L_q + \rho$. $\square$

Example 1.26

Suppose $\lambda = 2$ turns per second arrive at a prefill instance and the mean prefill work is $\mathbb{E}[S] = 0.3$ seconds. By Corollary 1.25 the prefill server is busy a fraction $\rho_P = 0.6$ of the time. If the observed mean number of turns waiting for it is $L_q = 0.9$, then $W_q = L_q/\lambda = 0.45$ seconds, the mean TTFT (with no wait for memory) is $W_q + \mathbb{E}[S] = 0.75$ seconds, and the mean number of turns at the prefill server is $L = \lambda \cdot 0.75 = 1.5$.

The scheduler's objective is the mean number of turns in the stations, for example $L = L_P + L_D$ for the prefill server and the decode batch. By Theorem 1.24 this equals the total delay accumulated per unit time, $\lambda \cdot (\text{mean sojourn time})$, so “reduce the mean number of turns” and “reduce the mean delay” are the same goal. The “price of a miss” of Lecture 3 is defined precisely as a change in L_P .

1.7 Exercises

Exercise 1.1. For independent $X_1 \sim \text{Exp}(\mu_1)$ and $X_2 \sim \text{Exp}(\mu_2)$, show that $\mathbb{P}(X_1 < X_2) = \mu_1/(\mu_1 + \mu_2)$, and that the event $\{X_1 < X_2\}$ is independent of $\min(X_1, X_2)$. *Hint:* integrate $\mathbb{P}(X_1 < X_2, \min > t)$ against the density of X_1 .

Exercise 1.2. Each time a session makes a tool call, it stays away from the server for a mean of Z seconds. If tool calls start at rate λ , show that the time-average number of sessions in a tool call is λZ . Does this hold whatever the distribution of the tool-call duration? *Hint:* take the tool-call stage alone as “the system” and apply Theorem 1.24.

Exercise 1.3. Remove **cache** κT from the program of Example 1.12. Show from Definition 1.9 that every turn then has $H_k = 0$, and compute $\mathbb{E}[S]$ from (1.10). Which of the conditions (1.12) becomes harder to meet?

Exercise 1.4. In a *colocated* replica, prefill and decode run on one instance with one pool of M bytes, and no transfer is needed. Write its ROUTE program with one step stage. The instance of the paper gives each iteration's budget first to the decoding turns and the rest to the oldest prefill; the step stage serves its jobs in their order of entry. Show that the two orders coincide when turns enter in the order they are admitted and a prefill may take all the budget left: after every iteration every job in its decode phase precedes every job in its prefill phase. Give a two-job example showing that a cap on a single prefill's tokens per iteration breaks this. *Hint:* a prefill that does not finish in an iteration leaves no budget behind it.

Lecture 2

Markovian Queues: Memory Slots, Processor Sharing, and Insensitivity

Lecture 1 dealt with facts, such as the arrival process and Little's law, that depend hardly at all on the details of the model. This lecture treats the case where the state of the system (the number of jobs present) is a continuous-time Markov chain. The stationary distribution can then be computed exactly. Three stations of the disaggregated system come out of it: the memory slots in front of each instance (a multi-server queue), and the decode batch and the KV link (processor sharing), with the key property of the latter, *insensitivity*.

2.1 Continuous-time Markov chains

Definition 2.1 (Continuous-time Markov chain)

A **generator** (Q -matrix) on a countable set $\mathcal{X}$ is a matrix $Q = (q(x, y))_{x, y \in \mathcal{X}}$ with $q(x, y) \geq 0$ for $x \neq y$, $q(x) := \sum_{y \neq x} q(x, y) < \infty$, and $q(x, x) = -q(x)$. A right-continuous process $(X_t)_{t \geq 0}$ with values in $\mathcal{X}$ is a **continuous-time Markov chain** (CTMC) with generator Q if, on entering state x , it stays there for an exponential time with mean $1/q(x)$ (independent of the past) and then jumps to y with probability $q(x, y)/q(x)$. $q(x, y)$ is the **transition rate** from x to y . The chain is **non-explosive** if infinitely many jumps in finite time have probability zero, and **irreducible** if every y can be reached from every x with positive probability.

The memorylessness of the exponential distribution (Proposition 1.15) is what makes this definition possible. If holding times were not exponential, how long the chain has already stayed would affect the future and the Markov property would fail. Intuitively, $q(x, y)$ means that the probability of moving from x to y in a short time h is $q(x, y)h + o(h)$.

Definition 2.2 (Stationary distribution, balance equations)

A probability distribution π on $\mathcal{X}$ with $\pi Q = 0$, that is, for all x ,

$$\pi(x) q(x) = \sum_{y \neq x} \pi(y) q(y, x), \quad (2.1)$$

is a **stationary distribution**, and (2.1) are the **global balance equations**. The left side is the probability flow out of x , the right side the probability flow into x . The stronger condition

$$\pi(x) q(x, y) = \pi(y) q(y, x) \quad (\forall x \neq y) \quad (2.2)$$

is called **detailed balance**.

Lemma 2.3 (Detailed balance [17])

A probability distribution satisfying detailed balance is stationary.

Proof. Summing (2.2) over $y \neq x$ gives $\pi(x) \sum_{y \neq x} q(x, y) = \sum_{y \neq x} \pi(y) q(y, x)$, which is (2.1). $\square$

We quote the next theorem without proof [17, Ch. 3]. It lets us read the stationary probability $\pi(x)$ as “the fraction of time the state is x ”.

Theorem 2.4 (Ergodic theorem [17])

If an irreducible, non-explosive CTMC has a stationary distribution π , then π is unique, the chain is positive recurrent, and for any initial state and any f with $\sum_x |f(x)|\pi(x) < \infty$, almost surely

$$\frac{1}{t} \int_0^t f(X_s) ds \longrightarrow \sum_x f(x) \pi(x) \quad (t \rightarrow \infty).$$

2.2 Birth–death processes and M/M/1

Definition 2.5 (Birth–death process)

A CTMC on $\mathcal{X} = \{0, 1, 2, \dots\}$ whose only nonzero transition rates are $q(n, n+1) = \lambda_n > 0$ ($n \geq 0$) and $q(n, n-1) = \mu_n > 0$ ($n \geq 1$) is a **birth–death process**. λ_n are the birth rates and μ_n the death rates.

In a queue, n is the number of jobs in the system. An arrival increases n by one and a completion decreases it by one, so if arrivals and completions happen on exponential clocks, the number of jobs is a birth–death process. It is known that the process is non-explosive when the birth rates are bounded.

Theorem 2.6 (Stationary distribution of a birth–death process [12])

A birth–death process has a stationary distribution if and only if

$$H := \sum_{n=0}^{\infty} \prod_{k=1}^n \frac{\lambda_{k-1}}{\mu_k} < \infty \quad (2.3)$$

(an empty product is 1), and then the stationary distribution is $\pi(n) = H^{-1} \prod_{k=1}^n \lambda_{k-1} / \mu_k$.

Proof. (Necessity and the form of π .) Suppose π satisfies (2.1). At state 0, $\pi(0)\lambda_0 = \pi(1)\mu_1$. For

$n \geq 1$, assume $\pi(n-1)\lambda_{n-1} = \pi(n)\mu_n$. Cancelling this term from the global balance at state n , $\pi(n)(\lambda_n + \mu_n) = \pi(n-1)\lambda_{n-1} + \pi(n+1)\mu_{n+1}$, gives $\pi(n)\lambda_n = \pi(n+1)\mu_{n+1}$. By induction, for all n

$$\pi(n)\lambda_n = \pi(n+1)\mu_{n+1}, \quad (2.4)$$

hence $\pi(n) = \pi(0) \prod_{k=1}^n \lambda_{k-1}/\mu_k$. For $\sum_n \pi(n) = 1$ we need $\pi(0) = 1/H > 0$, that is, $H < \infty$. (Sufficiency.) If $H < \infty$, the π defined above is a probability distribution, and (2.4) is exactly detailed balance (2.2), so π is stationary by Lemma 2.3. $\square$

Eq. (2.4) is called the **cut equation**: the flow across the boundary between $\{0, \dots, n\}$ and $\{n+1, \dots\}$ is the same in both directions.

Definition 2.7 (M/M/1 queue)

In Kendall notation, **M/M/1** is the queue with Poisson arrivals of rate λ (Definition 1.17), service times exponential with mean $1/\mu$ (independent of each other and of the arrivals), one server, and unlimited waiting room. Its load is $\rho = \lambda/\mu$ (Definition 1.6).

Proposition 2.8 (M/M/1 [12])

In M/M/1, the number of jobs N_t (including the one in service) is a birth–death process with $\lambda_n = \lambda$, $\mu_n = \mu$. If $\rho < 1$, the stationary distribution is geometric, $\pi(n) = (1 - \rho)\rho^n$, and

$$L = \frac{\rho}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\rho}{\mu(1 - \rho)}.$$

If $\rho \geq 1$, there is no stationary distribution. As $\rho \uparrow 1$, L , W and W_q all diverge to $+\infty$.

Proof. When $N_t = n \geq 1$, by memorylessness the time to the next arrival is $\text{Exp}(\lambda)$, the residual service time of the job in service is $\text{Exp}(\mu)$, and the two are independent. So the rate of $n \rightarrow n+1$ is λ and that of $n \rightarrow n-1$ is μ . In Theorem 2.6, $H = \sum_n \rho^n$ is finite only when $\rho < 1$, and then $H = 1/(1 - \rho)$. Hence $\pi(n) = (1 - \rho)\rho^n$ and $L = \sum_n n(1 - \rho)\rho^n = \rho/(1 - \rho)$. By Little’s law (Theorem 1.24), $W = L/\lambda = 1/(\mu - \lambda)$, and since the waiting time is the sojourn time minus the service time, $W_q = W - 1/\mu = \rho/(\mu(1 - \rho))$. $\square$

At $\rho = 0.9$ we get $L = 9$, and at $\rho = 0.99$, $L = 99$. This factor $1/(1 - \rho)$, which says that near capacity a small increase in mean load greatly increases the delay, reappears in the “price of a miss” of Lecture 3.

2.3 Multi-server queues and memory slots

An instance admits a turn only when its context fits in the instance’s memory. If every turn needs about the same memory, the pool is a set of identical *slots*, and the memory in front of an instance is a queue with several servers.

Definition 2.9 (M/M/m queue)

The **M/M/m** queue has Poisson arrivals of rate λ , m identical servers, service times that are independent $\text{Exp}(\mu)$ and independent of the arrivals, and an infinite waiting room served in arrival order; a customer waits only when all m servers are busy. Its **offered load** is $a = \lambda/\mu$, the mean number of busy servers; the load per server is a/m .

Proposition 2.10 (M/M/ m and the Erlang C formula [12])

The number in system of M/M/ m is a birth–death process with $\lambda_n = \lambda$ and $\mu_n = \min(n, m) \mu$. If $a < m$, its stationary distribution is

$$\pi(n) = \pi(0) \frac{a^n}{n!} \quad (n \leq m), \quad \pi(n) = \pi(0) \frac{a^n}{m! m^{n-m}} \quad (n \geq m),$$

with $\pi(0)^{-1} = \sum_{k=0}^{m-1} a^k/k! + \frac{a^m}{m!} \frac{m}{m-a}$. The probability that an arriving customer waits is the **Erlang C formula**

$$C(m, a) = \sum_{n \geq m} \pi(n) = \pi(0) \frac{a^m}{m!} \frac{m}{m-a},$$

and the mean wait is $W_q = C(m, a)/(\mu - \lambda)$. If $a \geq m$ there is no stationary distribution.

Proof. With n customers present, $\min(n, m)$ are in service. By memorylessness each completes at rate μ independently, so the first completion comes at rate $\min(n, m)\mu$ (Lemma 1.16), and arrivals come at rate λ . By Theorem 2.6, $\pi(n) \propto \prod_{k=1}^n \lambda/(\min(k, m)\mu)$, which is $a^n/n!$ for $n \leq m$ and $a^m/m! \cdot (a/m)^{n-m}$ for $n \geq m$. The sum is finite if and only if $a/m < 1$, and the geometric tail sums to $\frac{a^m}{m!} \frac{1}{1-a/m}$, which gives $\pi(0)$ and $C(m, a)$. By PASTA (Theorem 1.23) an arriving customer finds n present with probability $\pi(n)$, so it waits with probability $\sum_{n \geq m} \pi(n) = C(m, a)$. The number waiting is $(N - m)^+$; its mean is

$$L_q = \sum_{n \geq m} (n - m) \pi(n) = \pi(0) \frac{a^m}{m!} \sum_{j \geq 0} j \left(\frac{a}{m} \right)^j = \pi(0) \frac{a^m}{m!} \frac{a/m}{(1 - a/m)^2} = C(m, a) \frac{a}{m - a},$$

and Little’s law (Corollary 1.25(ii)) gives $W_q = L_q/\lambda = C(m, a) \frac{a}{\lambda(m-a)} = \frac{C(m, a)}{m\mu - \lambda}$. $\square$

For example, with $m = 4$ servers and offered load $a = 3$, $C(4, 3) \approx 0.51$: half of the arrivals wait, although only three quarters of the capacity is used; with $a = 2$ the share falls to 0.17. Waiting is a property of the whole distribution of busy servers, not of the mean load alone.

Memory slots. If every turn at an instance needs about c bytes of KV and the pool has M bytes, the pool holds $m = \lfloor M/c \rfloor$ slots, and the memory in front of the instance is an M/G/ m queue whose service time is the time a turn holds its slot: the prefill and the transfer on a prefill instance (m_P slots), the decode on a decode instance (m_D slots). Proposition 2.10 is exact when those holding times are exponential and approximate otherwise (the M/G/ m queue has no closed form; a common correction multiplies W_q by $(1 + \text{CV}^2)/2$ of the holding time). Two limits of the reading: with unequal contexts a turn needs several slots, and the station becomes a multi-resource queue with no closed form; and the blocks of the prefix cache on a prefill instance count as free, so the cache does not remove slots but loses entries whenever an admission needs their blocks.

2.4 Processor sharing and state-dependent capacity

So far each server has processed one job at a time. A decode instance does not, and neither does a link that concurrent transfers share.

Definition 2.11 (Work, processor sharing)

The **work** (service requirement) $S \geq 0$ of a job is the time the server needs to process it at full speed. Given a **capacity function** $\varphi : \{1, 2, \dots\} \rightarrow (0, \infty)$, the rule under which, when n jobs are present, the server processes $\varphi(n)$ units of work per second and splits it equally among the n jobs, so that each job's remaining work decreases at rate $\varphi(n)/n$, is called **processor sharing** (PS). $\varphi \equiv 1$ is classical PS; with Poisson arrivals and a general work distribution G we write **M/G/1-PS**. The load is $\rho = \lambda \mathbb{E}[S]$.

In LLM serving

A decode instance runs only decodes: one step advances every request in its batch by one token, so the m running requests share the instance equally, a PS station whose work is the decode demand D and whose load is $\lambda \mathbb{E}[D]$. Because the weights are read once for the whole batch, $\varphi(m)$ increases with m ; because the memory slots limit the batch, it saturates. The KV link is PS with constant capacity: concurrent transfers share its bandwidth B , and a transfer of T tokens is work $X = x_0 + \kappa T/B$.

Proposition 2.12 (PS with exponential work [7])

For a PS server with capacity φ , Poisson arrivals of rate λ , and exponential work with mean $\mathbb{E}[S]$, the number of jobs is a birth–death process with $\lambda_n = \lambda$ and $\mu_n = \varphi(n)/\mathbb{E}[S]$. Hence, with $F(n) := \prod_{k=1}^n \varphi(k)$, if $G(\rho) := \sum_{n \geq 0} \rho^n / F(n) < \infty$ then

$$\pi(n) = \frac{\rho^n / F(n)}{G(\rho)}, \quad \rho = \lambda \mathbb{E}[S]. \quad (2.5)$$

Proof. In state n , by memorylessness the remaining works of the n jobs are independent, each $\text{Exp}(1/\mathbb{E}[S])$, and each decreases at speed $\varphi(n)/n$. Hence each job's completion time is exponential with rate $\varphi(n)/(n\mathbb{E}[S])$, and the first of the n completions is exponential with rate $\varphi(n)/\mathbb{E}[S]$ (Lemma 1.16). The rest follows from Theorem 2.6, since $\prod_{k \leq n} \lambda \mathbb{E}[S] / \varphi(k) = \rho^n / F(n)$. $\square$

With $\varphi \equiv 1$, (2.5) is the geometric distribution of M/M/1. With exponential work, the service order does not change the distribution of the number of jobs.

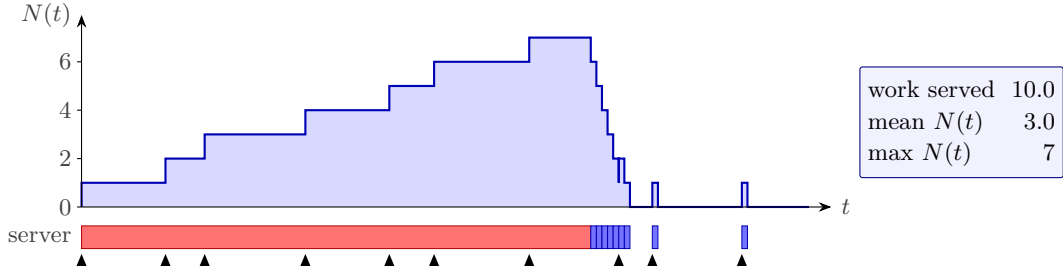
Example 2.13 (Saturating capacity)

Let $\varphi(m) = \min(m, c)$. Below c the throughput grows in proportion to the batch size, and it saturates at c (the simplest model of a memory that holds only c requests). This is the same birth–death process as M/M/ c with c servers, and by (2.5), $\pi(n) \propto \rho^n / n!$ for $n \leq c$ and $\pi(n) \propto \rho^n / (c! c^{n-c})$ for $n \geq c$. $G(\rho) < \infty$ if and only if $\rho < c = \sup \varphi$. If the load exceeds the saturated capacity, the system is unstable however large the batch grows.

The next theorem says that more expected demand never leaves fewer jobs. This sounds obvious, but it is not true in general: it depends on the discipline.

More work, fewer jobs. At a single FIFO server compare two work laws. Under law A a job takes 0.1 with probability 0.9 and 9.1 with probability 0.1; under law B every job takes 1.2, so B carries 20% more work. In Figure 2.1 the long job of law A holds the server while short jobs pile up behind it, and the number in system reaches 7; under law B it never exceeds 2. In steady state, with Poisson arrivals at $\lambda = 0.5$ ($\rho = 0.5$ for A, 0.6 for B), the Pollaczek–Khinchine formula of Lecture 3 gives a mean number in system of 2.57 under A and 1.05 under B. Under FIFO the wait is set by the second moment of the work, not its mean, so less expected demand

Law A: short jobs (0.1) with an occasional long one (9.1, probability 0.1)



Law B: every job takes 1.2 (20 % more work per job)

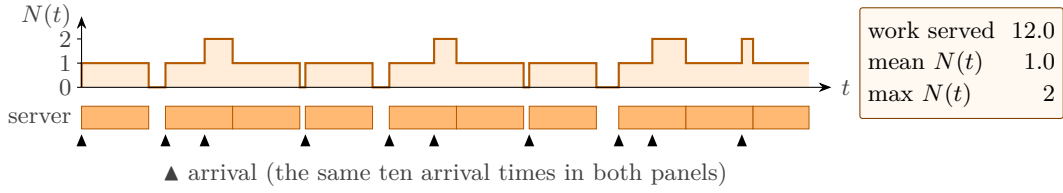


Figure 2.1: One sample path of each work law at a FIFO server, on the same arrival times. Top of each panel: the number in system $N(t)$; bottom: when the server works on each job (red: the long job of law A). *What to see:* law B does 20 % more work, yet law A keeps about three times as many jobs waiting, because one long job blocks everything behind it.

does not mean fewer jobs.

Under processor sharing a short job shares the server with the long one and leaves after about 0.2. With $\varphi \equiv 1$ the mean number in system is $\rho/(1-\rho)$ for every work law (Theorem 2.16 below): 1.0 under A and 1.5 under B, in the order of the load. Theorem 2.14 together with insensitivity makes this true for every capacity function φ . This contrast is exactly the one between the two kinds of instance: the prefill instance is a FIFO server, and the decode instance (like the link) is PS.

Theorem 2.14 (Monotonicity of the mean batch size)

Let φ be any positive function and R the radius of convergence of the power series $G(\rho) = \sum_n \rho^n / F(n)$. The mean $L(\rho) = \sum_n n \pi(n)$ of the stationary distribution (2.5) is differentiable on $(0, R)$ and

$$\frac{dL}{d\rho} = \frac{\text{Var}_\rho(N)}{\rho} \geq 0.$$

Hence L is nondecreasing in the load. If φ is nondecreasing, then $R = \sup_n \varphi(n)$.

Proof. Inside its radius of convergence a power series can be differentiated term by term, and the differentiated series has the same radius (a standard theorem of analysis). With $a_n = 1/F(n)$ we have $\rho G'(\rho) = \sum_n n a_n \rho^n$ and $\rho^2 G''(\rho) = \sum_n n(n-1) a_n \rho^n$, so

$$L = \frac{\rho G'}{G}, \quad \mathbb{E}_\rho[N(N-1)] = \frac{\rho^2 G''}{G}.$$

By the quotient rule,

$$\rho \frac{dL}{d\rho} = \frac{\rho G'}{G} + \frac{\rho^2 G''}{G} - \left(\frac{\rho G'}{G} \right)^2 = \mathbb{E}[N] + \mathbb{E}[N(N-1)] - \mathbb{E}[N]^2 = \text{Var}(N) \geq 0.$$

If φ is nondecreasing with $\varphi_\infty := \sup \varphi$, then $a_{n+1}/a_n = 1/\varphi(n+1) \rightarrow 1/\varphi_\infty$, so by the ratio test $R = \varphi_\infty$ ($R = \infty$ if $\varphi_\infty = \infty$). $\square$

For $\rho > 0$, N is not degenerate, so in fact L is strictly increasing. The theorem says, without knowing anything about the shape of φ , that “a policy that increases the mean demand does not shrink the batch”. Since φ must be calibrated from measurements, such shape-free conclusions matter.

2.5 Insensitivity

Proposition 2.12 assumed exponential work. But the decode demand is proportional to the output length, and there is no reason for it to be exponential. Remarkably, under PS the conclusion does not depend on the work distribution.

Definition 2.15 (Insensitivity)

A stationary quantity of a queue (here, the distribution of the number of jobs) is **insensitive** to the work distribution G if it depends on G only through its mean $\int x dG(x)$.

Theorem 2.16 (Insensitivity of PS [7, 2])

For a PS server with capacity φ and Poisson arrivals of rate λ , the stationary distribution of the number of jobs is insensitive to the work distribution and equals (2.5). We prove it here when the work is a finite mixture of exponentials, that is, $\text{Exp}(\mu_k)$ with probability α_k ($k = 1, \dots, K$, $\sum_k \alpha_k = 1$).

Proof. Think of each job as independently drawing a type k with probability α_k on arrival and then drawing its work from $\text{Exp}(\mu_k)$. The vector of job counts by type, $\mathbf{n} = (n_1, \dots, n_K)$ with $n = |\mathbf{n}| = \sum_k n_k$, is a CTMC. By thinning of the Poisson process (Proposition 1.19(ii)), type k arrives at rate $\lambda\alpha_k$ ($\mathbf{n} \rightarrow \mathbf{n} + e_k$), and as in the proof of Proposition 2.12 a single type- k job finishes at rate $\mu_k\varphi(n)/n$, so the rate of $\mathbf{n} \rightarrow \mathbf{n} - e_k$ is $n_k\mu_k\varphi(n)/n$. Let $\rho_k = \lambda\alpha_k/\mu_k$. We show that

$$\pi(\mathbf{n}) = B \frac{n!}{n_1! \cdots n_K!} \frac{\prod_k \rho_k^{n_k}}{F(n)}$$

satisfies global balance. If $n_k \geq 1$, then $\pi(\mathbf{n})/\pi(\mathbf{n} - e_k) = \frac{n}{n_k} \cdot \frac{\rho_k}{\varphi(n)}$, so

$$\pi(\mathbf{n}) \frac{n_k\mu_k\varphi(n)}{n} = \pi(\mathbf{n} - e_k) \rho_k\mu_k = \pi(\mathbf{n} - e_k) \lambda\alpha_k. \quad (2.6)$$

This says “the flow out of $\mathbf{n}$ by a type- k completion equals the flow into $\mathbf{n}$ by a type- k arrival”. Applying (2.6) at $\mathbf{n} + e_k$ and summing over k ,

$$\sum_k \pi(\mathbf{n} + e_k) \frac{(n_k + 1)\mu_k\varphi(n + 1)}{n + 1} = \sum_k \pi(\mathbf{n}) \lambda\alpha_k = \pi(\mathbf{n}) \lambda,$$

that is, “the flow out by arrivals equals the flow in by completions”. Adding the two gives global balance at $\mathbf{n}$. Splitting global balance in this way is called **partial balance**. Now, by the multinomial theorem,

$$\mathbb{P}(N = n) = \sum_{|\mathbf{n}|=n} \pi(\mathbf{n}) = \frac{B}{F(n)} \sum_{|\mathbf{n}|=n} \frac{n!}{\prod_k n_k!} \prod_k \rho_k^{n_k} = \frac{B \rho^n}{F(n)}, \quad \rho = \sum_k \rho_k = \lambda \sum_k \frac{\alpha_k}{\mu_k} = \lambda \mathbb{E}[S].$$

The normalisability condition is $G(\rho) < \infty$, with $B = 1/G(\rho)$. This is (2.5), and G involves only $\mathbb{E}[S]$. (Uniqueness follows from irreducibility and Theorem 2.4.) $\square$

Remark 2.17

Insensitivity for general work distributions is due to [7] and [2] (BCMP). One route to a proof is to extend the partial balance argument above to phase-type distributions and take limits, using the fact that phase-type distributions are dense, under weak convergence, among distributions on $[0, \infty)$. Here we only cite it. The same insensitivity holds for a **delay station** (an infinite-server station where every job immediately gets its own server, so nobody waits), which we use in Lecture 4.

Fairness of PS. The difference is visible job by job: in M/G/1-PS the mean sojourn time of a job with work x is $x/(1 - \rho)$, proportional to its own size and independent of the size distribution of the other jobs [11]. At the FIFO prefill instance, by contrast, a job's wait depends on the second moment of everyone's work (Lecture 3).

2.6 The price at a processor-sharing station

Proposition 2.18 (A processor-sharing station prices only work)

For a PS station with constant capacity C and load $\rho < C$, $L = \rho/(C - \rho)$. If a policy change increases the work of a fraction q_i of arrivals by $\Delta D_i \geq 0$, raising the load to $\rho' = \rho + \lambda \sum_i q_i \Delta D_i < C$, then exactly

$$\Delta L = \frac{C(\rho' - \rho)}{(C - \rho)(C - \rho')} = \frac{C - \rho}{C - \rho'} \lambda \sum_i q_i \Phi_i^D, \quad \Phi_i^D = \frac{C \Delta D_i}{(C - \rho)^2}.$$

In particular $\lambda \sum_i q_i \Phi_i^D \leq \Delta L$.

Proof. With $\varphi \equiv C$ we have $F(n) = C^n$, and (2.5) is geometric with ratio ρ/C , so $L = (\rho/C)/(1 - \rho/C) = \rho/(C - \rho)$. Therefore

$$\Delta L = \frac{\rho'}{C - \rho'} - \frac{\rho}{C - \rho} = \frac{\rho'(C - \rho) - \rho(C - \rho')}{(C - \rho)(C - \rho')} = \frac{C(\rho' - \rho)}{(C - \rho)(C - \rho')}.$$

Substituting $\rho' - \rho = \lambda \sum_i q_i \Delta D_i$ and multiplying and dividing by $(C - \rho)^2$ gives the second equality. Since $\rho' \geq \rho$, the leading factor is at least one. $\square$

$\Phi_i^D = \Delta D_i L'(\rho)$ is the first-order approximation, and the factor $(C - \rho)/(C - \rho')$ corrects for curvature. For example, with $C = 1$ and $\rho = 0.5 \rightarrow 0.6$, $\Delta L = 0.5$, the first-order term is 0.4, and the factor is 1.25.

The proposition applies to both PS stations of the disaggregated system. At the decode instance the added work is added decode demand (admitting a session, longer answers). At the link it is added transfer time: if a decode instance drops a paused session's KV, the next turn must receive its whole context over the link instead of only the new tokens, and $\Delta X_i = \kappa K_i/B$ for a context of K_i tokens. In both cases the price is $\Delta_i/(C - \rho)^2$ per unit of arrival rate, times C : it depends on the added work only through its mean. There is no head-of-line term, because at a PS station a long job does not hold up the short ones; the FIFO prefill instance of Lecture 3 has one.

Corollary 2.19

If a policy change alters the distribution of the decode work but not its mean $\mathbb{E}[D]$, the distribution of the number of jobs at the decode instance does not change.

Proof. By Theorem 2.16 the distribution depends only on $\rho_D = \lambda \mathbb{E}[D]$. □

In LLM serving

A prefix hit or miss changes only the prefill work on the prefill instance. It does not change the decode demand, and, since the decode instance receives the whole context either way, it does not change the transfer either. So by Corollary 2.19 a prefill miss costs nothing at the decode instance or on the link, and its whole price is paid at the FIFO prefill instance (Lecture 3). What the decode instance keeps, on the other hand, is priced at the link by Proposition 2.18, and what it admits is priced at its own batch.

2.7 Exercises

Exercise 2.1. For $\varphi(n) = n$ (each job has its own server, M/M/ ∞), show that (2.5) is the Poisson distribution with mean ρ , and check the formula $dL/d\rho = \text{Var}(N)/\rho$ of Theorem 2.14 directly. *Hint:* $F(n) = n!$, and a Poisson distribution has equal mean and variance.

Exercise 2.2. Show that in M/M/1-FIFO the sojourn time of an arriving job is $\text{Exp}(\mu - \lambda)$. *Hint:* by PASTA (Theorem 1.23) the number found on arrival is distributed as $(1 - \rho)\rho^n$, and conditionally the sojourn time is a sum of $n + 1$ independent $\text{Exp}(\mu)$ variables.

Lecture 3

The Prefill Instance as an M/G/1 Queue: the Price of a Miss

In Lecture 2 the service times were exponential, so the state of the queue was summarised by a single number, the number of customers. Prefill work is not exponential. It is a general distribution that mixes short appends (hits) with long recomputations (misses). A prefill instance runs prefills only, one after another in arrival order, so it is a single first-in first-out server whose service time is the prefill work. The goal of this lecture is to compute its mean waiting time (the Pollaczek–Khinchine formula), and then to differentiate it to obtain the price of a miss: the total delay one prefix miss adds. The key message is a single one. *The mean wait of a FIFO queue depends not only on the mean of the work but also on its second moment.*

3.1 The M/G/1 queue and the residual service time

Definition 3.1 (M/G/1 FIFO queue)

An **M/G/1 queue** is a single-server queue with the following properties.

- (a) Arrivals form a Poisson process of rate $\lambda > 0$ (Definition 1.17).
- (b) The **service time** (work) S_k of the k -th customer is i.i.d. with an arbitrary distribution G (“General”) and independent of the arrival process. S denotes a generic random variable with law G , and we assume $\mathbb{E}[S] < \infty$.
- (c) There is one server, the waiting room is infinite, and the service discipline is **FIFO** (first in, first out): when the server becomes free, it serves the earliest waiting customer to completion without interruption.

The load is $\rho = \lambda \mathbb{E}[S]$ (Definition 1.6). The **waiting time in queue** W_q of a customer is the time from arrival to the start of service, and the **sojourn time** is $W_q + S$. L_q and L are the time averages of the number of customers in the waiting room and in the whole system (waiting room and server), respectively.

In LLM serving

A prefill instance is an M/G/1 queue in this sense, except for the Poisson assumption (Remark 3.14). A turn is a customer; its work S is its prefill time on the instance, which runs prefills only and at full capacity; the load is $\rho_P = \lambda \mathbb{E}[S]$; and the time to first token is the sojourn time $W_q + S$. We write ρ for ρ_P in this lecture.

Unlike the exponential distribution, a general service distribution is not memoryless (Proposition 1.15). How much longer a customer who has already been served for 3 seconds will take

depends on the information “3 seconds”. The wait of an arriving customer therefore requires the quantity “how much service is left for the customer now in service”.

Definition 3.2 (Residual service time)

If the server is serving a customer at time t , let $R(t)$ be the remaining service time of that customer; if the server is idle, set $R(t) = 0$. $R(t)$ is called the **residual service time**.

The sample path of $R(t)$ is a sawtooth. At the moment service of customer k begins, it jumps up to S_k ; it then decreases with slope -1 , and when it reaches 0 the next service begins or the server becomes idle. Hence the contribution of a single customer k to $\int R(t) dt$ is the area $S_k^2/2$ of a triangle with base and height S_k .

Lemma 3.3 (Time average of the residual service time [12])

In an M/G/1 queue, suppose $\mathbb{E}[S^2] < \infty$ and that the number $D(T)$ of customers who have completed service by time T satisfies $D(T)/T \rightarrow \lambda$ (a.s.) (stability; this holds if $\rho < 1$). Then almost surely

$$\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T R(t) dt = \frac{\lambda \mathbb{E}[S^2]}{2} = \rho \cdot \frac{\mathbb{E}[S^2]}{2\mathbb{E}[S]}.$$

In particular, averaged only over the time the server is busy, the mean residual service time is $\mathbb{E}[S^2]/(2\mathbb{E}[S])$.

Proof. Let $A(T)$ be the number of arrivals in $[0, T]$. The triangles of the customers who completed service in $[0, T]$ lie entirely in $[0, T]$, and every customer whose service has started arrived before T , so

$$\sum_{k=1}^{D(T)} \frac{S_k^2}{2} \leq \int_0^T R(t) dt \leq \sum_{k=1}^{A(T)} \frac{S_k^2}{2}.$$

(Under FIFO the service order equals the arrival order, so the first $D(T)$ customers are exactly those who have completed service.) By the strong law of large numbers $\frac{1}{n} \sum_{k \leq n} S_k^2 \rightarrow \mathbb{E}[S^2]$; for a Poisson process $A(T)/T \rightarrow \lambda$, and by assumption $D(T)/T \rightarrow \lambda$. Hence

$$\frac{1}{T} \sum_{k=1}^{D(T)} \frac{S_k^2}{2} = \frac{D(T)}{T} \cdot \frac{1}{D(T)} \sum_{k=1}^{D(T)} \frac{S_k^2}{2} \rightarrow \lambda \cdot \frac{\mathbb{E}[S^2]}{2},$$

and the upper sum has the same limit. The squeeze gives the first equality. The second equality is $\lambda = \rho/\mathbb{E}[S]$. The fraction of time the server is busy is ρ (Corollary 1.25) and $R = 0$ during idle time, so the average over busy time is $\lambda \mathbb{E}[S^2]/(2\rho) = \mathbb{E}[S^2]/(2\mathbb{E}[S])$. $\square$

We write $\text{CV}^2 = \text{Var}[S]/\mathbb{E}[S]^2$ for the **squared coefficient of variation** of the work, so that the mean residual over busy time is $\mathbb{E}[S^2]/(2\mathbb{E}[S]) = \frac{\mathbb{E}[S]}{2}(1 + \text{CV}^2)$.

3.2 The Pollaczek–Khinchine formula

Theorem 3.4 (Pollaczek–Khinchine mean formula [18, 8])

In an M/G/1 FIFO queue let $\rho < 1$ and $\mathbb{E}[S^2] < \infty$. Then in steady state

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}, \quad L = \rho + \frac{\lambda^2 \mathbb{E}[S^2]}{2(1 - \rho)}. \quad (3.1)$$

Steady-state assumption. The proof takes the following facts as given. If $\rho < 1$, the queue is stable, the waiting time $W_q^{(k)}$ of the k -th customer converges in distribution, and the limit law has a finite mean (when $\mathbb{E}[S^2] < \infty$). Moreover the customer averages and the time averages exist almost surely. These are standard results obtained by analysing the Lindley recursion $W_q^{(k+1)} = (W_q^{(k)} + S_k - \tau_{k+1})^+$ (τ the interarrival times); see [1, Ch. 3, 10] and [12, Ch. 5]. The proof below accepts this existence and computes the *value* of the limit.

Proof. Consider a customer who has just arrived in steady state (the “tagged customer”). Under FIFO this customer waits for (1) the residual service time of the customer in service and (2) the service times of the N_q customers already in the waiting room:

$$W_q = R + \sum_{j=1}^{N_q} S_{(j)},$$

where R is the residual service time at the arrival instant and $S_{(j)}$ is the service time of the j -th waiting customer.

Mean of (1). Poisson arrivals see time averages (PASTA, Theorem 1.23). So the law of R seen at the arrival instant is the time-stationary law of $R(t)$, and by Lemma 3.3 $\mathbb{E}[R] = \lambda \mathbb{E}[S^2]/2$.

Mean of (2). The waiting customers have not started service yet. Under FIFO the state of the queue up to the arrival of the tagged customer is determined only by the arrival times and the service times of the customers who have already started service. Hence the service times $S_{(1)}, S_{(2)}, \dots$ of the customers not yet started can be regarded as i.i.d. G random variables independent of N_q . Therefore $\mathbb{E}[\sum_{j \leq N_q} S_{(j)}] = \sum_n \mathbb{P}(N_q = n) n \mathbb{E}[S] = \mathbb{E}[N_q] \mathbb{E}[S]$. By PASTA again, $\mathbb{E}[N_q]$ equals the time average L_q of the number in the waiting room.

Combining. Applying Little’s law (Theorem 1.24) to the waiting room alone as the “system” gives $L_q = \lambda \mathbb{E}[W_q]$. Therefore

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2} + \lambda \mathbb{E}[W_q] \mathbb{E}[S] = \frac{\lambda \mathbb{E}[S^2]}{2} + \rho \mathbb{E}[W_q].$$

Since $\mathbb{E}[W_q] < \infty$, subtracting $\rho \mathbb{E}[W_q]$ from both sides and dividing by $1 - \rho > 0$ gives the first formula. The second formula is Little’s law applied to the whole system: $L = \lambda(\mathbb{E}[W_q] + \mathbb{E}[S]) = \rho + \lambda \mathbb{E}[W_q]$. $\square$

Corollary 3.5 (Variance increases the wait [12])

Under the assumptions of Theorem 3.4,

$$\mathbb{E}[W_q] = \frac{\rho}{1 - \rho} \cdot \frac{1 + \text{CV}^2}{2} \cdot \mathbb{E}[S]. \quad (3.2)$$

Hence, of two work laws with the same λ and the same mean $\mathbb{E}[S]$ (so the same load $\rho < 1$), the one with the larger variance waits strictly longer. In particular:

- (i) M/M/1 ($\text{CV}^2 = 1$): $\mathbb{E}[W_q] = \rho \mathbb{E}[S]/(1 - \rho)$, in agreement with Proposition 2.8.
- (ii) M/D/1 (deterministic service, $\text{CV}^2 = 0$): $\mathbb{E}[W_q] = \rho \mathbb{E}[S]/(2(1 - \rho))$, exactly half of M/M/1 at the same load.

Proof. Substituting $\mathbb{E}[S^2] = \mathbb{E}[S]^2(1 + \text{CV}^2)$ into Eq. (3.1) and using $\lambda \mathbb{E}[S] = \rho$ gives Eq. (3.2). With ρ and $\mathbb{E}[S]$ fixed, the right side is strictly increasing in CV^2 . (i) and (ii) substitute $\text{CV}^2 = 1, 0$. For an exponential law with mean $1/\mu$ this is $\rho/(\mu(1 - \rho))$, the M/M/1 waiting time. $\square$

Remark 3.6

Contrast this with the PS queue of Lecture 2 (Definition 2.11). The mean number in an M/G/1-PS queue is $\rho/(1-\rho)$, insensitive to the work law (Theorem 2.16), while the mean number in FIFO, Eq. (3.1), is sensitive to the law through $\mathbb{E}[S^2]$. A single long job blocks every customer behind it under FIFO, but not under PS, where all customers share the server.

3.3 Mixtures of hits and misses

The prefix cache of the prefill instance decides whether each turn is a hit or a miss, and hence sets the law of the prefill work.

Definition 3.7 (Hit/miss mixture work)

Let S^{hit} and S^{miss} be square-integrable nonnegative random variables with $S^{\text{miss}} \geq S^{\text{hit}}$ (a.s.). At **hit rate** $h \in [0, 1]$ the work S_h has the law of S^{hit} with probability h and the law of S^{miss} with probability $1 - h$ (the choice is made by an independent coin toss).

Corollary 3.8 (A higher hit rate lowers the PK wait)

Let $m_1(h) = \mathbb{E}[S_h]$, $m_2(h) = \mathbb{E}[S_h^2]$ and $\rho(h) = \lambda m_1(h)$.

- (i) m_1 , m_2 and ρ are affine and nonincreasing in h .
- (ii) If $\rho(h_0) < 1$, then $\rho(h) < 1$ for all $h \in [h_0, 1]$, and $W(h) = \lambda m_2(h)/(2(1 - \rho(h)))$ is nonincreasing on $[h_0, 1]$.

Proof. (i) By the law of total expectation $m_k(h) = h\mathbb{E}[(S^{\text{hit}})^k] + (1-h)\mathbb{E}[(S^{\text{miss}})^k]$, $k = 1, 2$, and its slope in h is $\mathbb{E}[(S^{\text{hit}})^k] - \mathbb{E}[(S^{\text{miss}})^k] \leq 0$ (since $0 \leq S^{\text{hit}} \leq S^{\text{miss}}$, k -th powers preserve the order). (ii) Since ρ is nonincreasing, $\rho(h) \leq \rho(h_0) < 1$. If $h_0 \leq h \leq h'$, then $0 \leq \lambda m_2(h') \leq \lambda m_2(h)$ and $1 - \rho(h') \geq 1 - \rho(h) > 0$, so the numerator decreases and the denominator increases, giving $W(h') \leq W(h)$. $\square$

3.4 The price of a miss

Corollary 3.8 gives only a direction. A scheduler needs a *magnitude*. If the prefill instance drops the cached prefix of a particular program i , so that its next turn is a miss, by how much does the delay of the whole instance grow?

Definition 3.9 (Mean number at the prefill instance and policy change)

Model the prefill instance as an M/G/1 FIFO queue with arrival rate λ , work S ($m_1 = \mathbb{E}[S]$, $m_2 = \mathbb{E}[S^2] < \infty$) and load $\rho = \lambda m_1 < 1$. Let $W = \lambda m_2 / (2(1 - \rho))$ be the PK wait and L_P the mean number at the instance (waiting and in service). By Little's law

$$L_P = \lambda(W + m_1) = \rho + \frac{\lambda^2 m_2}{2(1 - \rho)}.$$

For programs (classes) $i = 1, \dots, k$, a **policy change** turns a fraction $q_i \geq 0$ of arrivals ($\sum_i q_i \leq 1$) from hits with work $S_i^{\text{hit}} \geq 0$ into misses with work $S_i^{\text{miss}} = S_i^{\text{hit}} + \Delta S_i$, $\Delta S_i \geq 0$. The moments of the work become

$$m'_1 = m_1 + A, \quad A = \sum_i q_i \Delta S_i, \quad m'_2 = m_2 + B, \quad B = \sum_i q_i ((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2),$$

and the new load is $\rho' = \rho + \lambda A$. We write L'_P for the new mean number and $\Delta L_P = L'_P - L_P$ for the change.

Theorem 3.10 (The price of a miss)

In the setting of Definition 3.9, suppose $\rho' < 1$ and define

$$\Phi_i = \Delta S_i + \frac{\lambda((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2)}{2(1 - \rho)} + \frac{\lambda W \Delta S_i}{1 - \rho}. \quad (3.3)$$

Then

$$\lambda \sum_i q_i \Phi_i \leq \Delta L_P \leq \frac{1 - \rho}{1 - \rho'} \lambda \sum_i q_i \Phi_i.$$

Proof. Write $u = 1 - \rho > 0$. Then $1 - \rho' = u - \lambda A > 0$, and since $A \geq 0$, $0 < u - \lambda A \leq u$. Applying the PK formula (Theorem 3.4) to the new work,

$$\Delta L_P = \lambda A + \frac{\lambda^2}{2} \left(\frac{m_2 + B}{u - \lambda A} - \frac{m_2}{u} \right) = \lambda A + \frac{\lambda^2}{2} \cdot \frac{u(m_2 + B) - (u - \lambda A)m_2}{u(u - \lambda A)} = \lambda A + \frac{Y}{u(u - \lambda A)},$$

where $Y = \frac{\lambda^2}{2}(uB + \lambda A m_2) \geq 0$. On the other hand, taking the q_i -weighted sum of Eq. (3.3) and using $W = \lambda m_2 / (2u)$,

$$\lambda \sum_i q_i \Phi_i = \lambda A + \frac{\lambda^2 B}{2u} + \frac{\lambda^2 W A}{u} = \lambda A + \frac{\lambda^2}{2u^2} (uB + \lambda A m_2) = \lambda A + \frac{Y}{u^2}.$$

Lower bound. Since $u - \lambda A \leq u$, $Y/(u(u - \lambda A)) \geq Y/u^2$, and hence $\Delta L_P \geq \lambda \sum_i q_i \Phi_i$.

Upper bound. A direct computation gives

$$\frac{u}{u - \lambda A} \left(\lambda A + \frac{Y}{u^2} \right) - \Delta L_P = \lambda A \left(\frac{u}{u - \lambda A} - 1 \right) + \frac{Y}{u(u - \lambda A)} - \frac{Y}{u(u - \lambda A)} = \frac{(\lambda A)^2}{u - \lambda A} \geq 0.$$

Since $u/(u - \lambda A) = (1 - \rho)/(1 - \rho')$, the upper bound holds. $\square$

Reading the three terms. Φ_i is measured in seconds. Misses of program i occur at rate λq_i , so by Little's law $\lambda q_i \Phi_i$ is an increase in the mean number, and Φ_i is “the total waiting time, summed over all turns, that one miss adds”.

(1) ΔS_i : the miss itself takes longer to serve (its own cost).

- (2) $\lambda((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2)/(2(1 - \rho))$: turns that arrive while the miss is being served wait for all of it. This comes from the second moment in the PK formula, and its size is the difference of the *squares* of the work. It is the cost imposed on other customers, the **externality** of the economics of queues; in these notes we call it the **head-of-line term**.
- (3) $\lambda W \Delta S_i / (1 - \rho)$: the extra work raises the load, which stretches every wait by the factor $1/(1 - \rho)$ (the load term).

Since $(S^{\text{miss}})^2 - (S^{\text{hit}})^2 = (S^{\text{miss}} + S^{\text{hit}})\Delta S$,

$$\frac{(2)}{(1)} = \frac{\lambda(S_i^{\text{miss}} + S_i^{\text{hit}})}{2(1 - \rho)}, \quad \frac{(2)}{(3)} = \frac{S_i^{\text{miss}} + S_i^{\text{hit}}}{2W}.$$

The first ratio is of the order of “the number of turns that arrive while one miss is served”. When it is small, the price is close to ΔS_i ; when it is large, the head-of-line term dominates. Once the mean wait W exceeds half the sum of the miss and hit work, the load term exceeds the head-of-line term.

Example 3.11 (At moderate load the head-of-line term dominates)

In seconds, let $\lambda = 1.25$, $m_1 = 0.4$, $m_2 = 0.5$, so $\rho = 0.5$ and $W = 0.625$. Let a program have hit work $S^{\text{hit}} = 0.2$ and miss work $S^{\text{miss}} = 2.0$ ($\Delta S = 1.8$). The three terms are

$$\Delta S = 1.8, \quad \frac{1.25(4 - 0.04)}{2 \cdot 0.5} = 4.95, \quad \frac{1.25 \cdot 0.625 \cdot 1.8}{0.5} = 2.8125,$$

so $\Phi = 9.5625$ seconds. One miss creates more than five times its own extra service in total waiting, and most of it is paid by the turns queued behind it. The two ratios are 2.75 and 1.76. If a fraction $q = 0.01$ of this program’s turns turn into misses, then $\rho' = 0.5225$, and the exact change $\Delta L_P \approx 0.1241$ lies in the interval $[0.1195, 0.1252]$ of the theorem.

Corollary 3.12 (Divergence near saturation)

Fix the work law ($m_1 > 0$, $m_2 > 0$) and $\Delta S_i > 0$, and let $\lambda \uparrow 1/m_1$ (that is, $\rho \uparrow 1$). Then $\Phi_i \rightarrow \infty$.

Proof. Put $\rho = 1 - \varepsilon$ with $0 < \varepsilon \leq 1/2$, so $\lambda = (1 - \varepsilon)/m_1 \geq 1/(2m_1)$. The first two terms of Φ_i are nonnegative, and the third is $\lambda W \Delta S_i / \varepsilon = \lambda^2 m_2 \Delta S_i / (2\varepsilon^2) \geq m_2 \Delta S_i / (8m_1^2 \varepsilon^2)$, which diverges as $\varepsilon \rightarrow 0$. $\square$

Corollary 3.13 (Monotonicity in the context length)

Let the prefill cost be $P(n, K) = an + bn(K + n/2)$ ($a, b \geq 0$; the cost of computing n tokens on top of a prefix of length K). A hit costs $S^{\text{hit}} = P(n, K)$ and a miss, which recomputes the whole context, costs $S^{\text{miss}} = P(K + n, 0)$. Then $\Delta S = aK + bK^2/2$, and with λ, ρ, W and the number n of appended tokens fixed, Φ is nondecreasing in $K \geq 0$.

Proof. $P(K + n, 0) - P(n, K) = a(K + n) + \frac{b}{2}(K + n)^2 - an - bnK - \frac{b}{2}n^2 = aK + \frac{b}{2}K^2$. Both ΔS and $S^{\text{hit}} = an + bnK + bn^2/2$ are nonnegative and nondecreasing in K , and so is $S^{\text{miss}} + S^{\text{hit}} = 2S^{\text{hit}} + \Delta S$. The three terms of Eq. (3.3) are nonnegative constants times ΔS , $(S^{\text{miss}} + S^{\text{hit}})\Delta S$ and ΔS , respectively, so all are nondecreasing in K . $\square$

In LLM serving

A prefill instance serves pending prefills in arrival order; if it splits a long prefill into chunks, the chunks do not reorder the queue, so a prefill that arrives behind a long miss cannot start until every chunk of that miss is done. As long as the order is FIFO, the head-of-line term (2) remains, whatever the chunk size. And since the first token is produced by the prefill instance, neither the KV transfer to the decode instance nor the decode itself enters the time to first token: Φ_i is the whole effect of a miss on TTFT, summed over all turns.

In LLM serving

At the same load and with the same append, Corollary 3.13 says that the price of a miss is larger for a program with a longer context. But what the scheduler compares is $p_i\Phi_i/c_i$, divided by the memory c_i and weighted by the resume probability p_i , and that order can flip with the load, because the load changes the weights of the three terms of Φ_i .

Remark 3.14 (Limits of the M/G/1 assumption)

M/G/1 assumes an infinite customer population. In practice a prefill instance serves a few dozen live sessions at a time, and a session waiting for prefill cannot create its next turn, so arrivals slow down when the queue is long. The finite population model of Lecture 4 (Proposition 4.6) captures this effect exactly and shows that, for exponential work, the wait of the open formula is at least the finite-population wait at the same utilisation. In that case the W of this lecture is a conservative (upper) estimate. For general work the relation between the two waits is a question for measurement.

3.5 Exercises

Exercise 3.1. Show that the mean residual service time over busy time, $\mathbb{E}[S^2]/(2\mathbb{E}[S])$, is always at least $\mathbb{E}[S]/2$, with equality only when S is deterministic. Also check that it equals $\mathbb{E}[S]$ when S is exponential. *Hint:* $\mathbb{E}[S^2] = \mathbb{E}[S]^2 + \text{Var}[S]$.

Exercise 3.2. With the numbers of Example 3.11 and $q = 0.01$, compute ΔL_P exactly and check $0.1195 \leq \Delta L_P \leq 0.1252$. How does the ratio of the upper to the lower bound change as q grows? *Hint:* substitute $m_1 + A$ and $m_2 + B$ into $L_P = \lambda m_1 + \lambda^2 m_2 / (2(1 - \lambda m_1))$.

Lecture 4

Closed Systems and Memory: Finite Populations, MVA, Campbell's Theorem, Eviction

All models so far were **open systems**. Customers arrive from an infinite outside population, leave after service, and the arrival rate does not change when the queue is long. Agent sessions are different. A session submits a turn, which passes the prefill instance, the link and a decode instance, runs a tool on the response, and only then submits its next turn. A waiting session cannot create a new turn, so the longer the queue, the fewer the arrivals. This lecture treats this *closed* structure (Section 4.1), computes the mean memory held by sessions (Section 4.2), and solves the combinatorial optimisation problem of what to drop when memory runs short (Section 4.3).

4.1 Closed systems and mean value analysis

Definition 4.1 (Delay station, think time)

A **delay station** or **infinite-server station** is a station with infinitely many servers, so that an arriving customer is served at once without waiting. A customer's sojourn time equals its own service time and does not depend on other customers. In a closed system the time spent at a delay station is called the **think time**, and its mean is written Z .

Definition 4.2 (Closed system, finite-source model)

A **closed system** is a system in which a fixed number N of customers circulate among stations forever, with no entry from or exit to the outside. The **finite-source model** or machine-repairman model $M/M/1//N$ is the closed system made of two stations:

- (a) a delay station with exponential think times of mean Z ,
- (b) a FIFO single server with exponential service times of mean $1/\mu$.

Each customer (session) submits a turn to the server when it finishes thinking, and starts thinking again when its service is done. All think times and service times are mutually independent.

In LLM serving

Sessions are the customers of the closed loop prefill instance $\rightarrow$ link $\rightarrow$ decode instance $\rightarrow$ tool $\rightarrow$ prefill instance. Seen from the prefill instance, the server is the prefill server and everything else (transfer, decode, tool call and user response) is the “think time”. A session waiting for prefill cannot submit its next turn. A memory pool holds a few million tokens and agent contexts reach hundreds of thousands of tokens, so an instance serves only about ten live sessions at a time; at this scale the $N = \infty$ approximation can be badly wrong.

Let $X(t)$ be the number of turns at the server (waiting or in service) at time t . There are $N - X(t)$ thinking sessions, each finishing its think time independently at rate $1/Z$ (Proposition 1.15), so X is a birth–death process on $\{0, 1, \dots, N\}$ with birth rates $\lambda_n = (N - n)/Z$ and death rates $\mu_n = \mu$ ($n \geq 1$). By Theorem 2.6 the stationary law is

$$\pi_N(n) = \frac{1}{G_N} \prod_{k=0}^{n-1} \frac{(N - k)/Z}{\mu} = \frac{1}{G_N} \frac{N!}{(N - n)!} \left(\frac{1}{\mu Z} \right)^n, \quad n = 0, 1, \dots, N, \quad (4.1)$$

with normalising constant G_N . The state space is finite, so no stability condition is needed.

In an open system PASTA (Theorem 1.23) told us that an arriving customer sees the time-average law. In a closed system arrivals are not Poisson, and an arriving customer does not see the time-average law. Instead the following holds.

Theorem 4.3 (Arrival theorem [19, 13])

In M/M/1// N in steady state, the law of the number of turns at the server seen by an arriving turn (excluding itself), that is, the long-run fraction $a_N(n)$ of arrivals that occur in state n , equals the time-stationary law of the same system with $N - 1$ sessions: $a_N(n) = \pi_{N-1}(n)$, $n = 0, \dots, N - 1$.

Proof. In state n arrivals occur at rate $(N - n)/Z$. By the ergodic theorem (Theorem 2.4) and the law of large numbers for jump counts, the number of arrivals in state n during $[0, t]$, divided by t , converges almost surely to $\pi_N(n)(N - n)/Z$. Hence

$$a_N(n) = \frac{(N - n) \pi_N(n)}{\sum_{m=0}^N (N - m) \pi_N(m)}.$$

By (4.1), for $n \leq N - 1$ we have $(N - n) \frac{N!}{(N - n)!} = N \frac{(N - 1)!}{(N - 1 - n)!}$, and for $n = N$, $(N - n) \pi_N(n) = 0$. Therefore $a_N(n) \propto \frac{(N - 1)!}{(N - 1 - n)!} (\mu Z)^{-n} \propto \pi_{N-1}(n)$, and since both are probability distributions they are equal. $\square$

Remark 4.4 (Back to PASTA as $N \rightarrow \infty$)

Fix $\lambda > 0$ and $\mu > \lambda$, and let the think time grow with the population, $Z = N/\lambda$, so that the total arrival rate when all sessions think stays λ . Then $\pi_N(n) \propto \frac{N!}{(N - n)! N^n} \rho^n$ with $\rho = \lambda/\mu$, and $\frac{N!}{(N - n)! N^n} = \prod_{k=0}^{n-1} (1 - k/N)$ lies in $[0, 1]$ and tends to 1. By dominated convergence (with dominating sequence ρ^n) both π_N and π_{N-1} converge to the M/M/1 law $(1 - \rho)\rho^n$. So in the limit the law seen by an arriving turn equals the time-average law: this is PASTA. For a finite N the arrival theorem measures exactly how far the closed system is from that limit.

The arrival theorem lets us solve a closed system by recursion on the number of sessions. With n sessions, let Q_n be the stationary mean number of turns at the server, R_n the mean response time of a turn (wait plus service), and X_n the throughput (turns completed per unit time).

Theorem 4.5 (Mean value analysis [19])

In M/M/1// n , $Q_0 = 0$ and for $n \geq 1$

$$R_n = \frac{1 + Q_{n-1}}{\mu}, \quad X_n = \frac{n}{Z + R_n}, \quad Q_n = X_n R_n.$$

Hence $Q_n = n(1 + Q_{n-1})/(\mu Z + 1 + Q_{n-1})$, the server utilisation is $\rho_n = X_n/\mu = n/(\mu Z + 1 + Q_{n-1})$, and the mean waiting time of a turn (excluding service) is $W_n = Q_{n-1}/\mu$.

Proof. By Theorem 4.3 an arriving turn sees on average Q_{n-1} turns ahead of it. Under FIFO all of them must finish before its service begins. By memorylessness (Proposition 1.15) the remaining time of the turn in service again has mean $1/\mu$, and each waiting turn has mean $1/\mu$, so the mean wait is Q_{n-1}/μ , and the response time is this plus the turn's own service $1/\mu$, which is R_n . One cycle of a session takes $Z + R_n$ on average. Applying Little's law (Theorem 1.24) to the whole system (all n sessions, always n customers) gives $n = X_n(Z + R_n)$, and applying it to the server gives $Q_n = X_n R_n$. The utilisation is $\rho_n = X_n/\mu$ by Corollary 1.25, and substitution gives the remaining formulas. $\square$

Proposition 4.6 (Finite population)

Let $c = \mu Z > 0$ and define Q_n by the recursion of Theorem 4.5.

- (i) Q_n is nondecreasing in n , $0 \leq Q_n \leq n$, and Q_n is nonincreasing in c . Moreover $\rho_N < 1$ for all $N \geq 1$.
- (ii) $W_N \leq \rho_N/(\mu(1 - \rho_N))$. The right side is the open M/M/1 waiting time at the same utilisation (Proposition 2.8).

Proof. Put $g_c(q) = (1 + q)/(c + 1 + q)$, so that $Q_n = n g_c(Q_{n-1})$. On $q \geq 0$, g_c is nondecreasing in q , nonincreasing in c , and $0 < g_c \leq 1$.

(i) $0 \leq Q_n \leq n$ follows by induction from $g_c \leq 1$. $Q_1 = g_c(0) \geq 0 = Q_0$, and if $Q_n \geq Q_{n-1}$ then $Q_{n+1} = (n+1)g_c(Q_n) \geq n g_c(Q_{n-1}) = Q_n$. For $c \leq c'$, assuming $Q_{n-1}(c') \leq Q_{n-1}(c)$, $Q_n(c') = n g_{c'}(Q_{n-1}(c')) \leq n g_{c'}(Q_{n-1}(c)) \leq n g_c(Q_{n-1}(c)) = Q_n(c)$. $\rho_N < 1$ is equivalent to $Q_{N-1} > N - 1 - c$, so it suffices to show $Q_n > n - c$ for all $n \geq 0$ by induction. For $n = 0$ this is $0 > -c$, and a computation gives

$$Q_{n+1} - (n + 1 - c) = \frac{c(Q_n + c - n)}{c + 1 + Q_n} > 0.$$

(ii) Since $\rho_N < 1$, the inequality is equivalent to $Q_{N-1}(1 - \rho_N) \leq \rho_N$, that is, $Q_{N-1} \leq \rho_N(1 + Q_{N-1}) = Q_N$, which is (i). $\square$

Example 4.7

With $N = 2$ and $Z = 2/\mu$ ($c = 2$), $Q_1 = 1/3$, $\rho_2 = 2/(3 + 1/3) = 3/5$ and $W_2 = 1/(3\mu)$. The open formula at the same utilisation gives $\rho_2/(\mu(1 - \rho_2)) = 3/(2\mu)$, which is 4.5 times the actual wait.

Remark 4.8

For exponential work and at the same utilisation, the open formula (Theorem 3.4) and the price of a miss built on it (Theorem 3.10) overestimate the wait (Proposition 4.6(ii)). For general work this comparison has not been proved. Moreover, at most N turns exist, so $0 \leq L_P \leq N$, and no policy change can raise the mean number of turns at the server by more than $N - L_P$. This cap does not depend on the work law.

Remark 4.9

We assumed exponential think times, but the results depend only on their mean Z . The delay station is insensitive like the PS station (Theorem 2.16), and by the BCMP theorem a network of delay stations and PS stations has a product-form stationary law [2]. The service times of a FIFO server do not generalise in this way.

4.2 M/G/∞ and Campbell's theorem

We now count memory. Live sessions do not wait for each other and each keeps its own KV, so from the point of view of memory the sessions are customers of a delay station. At the prefill instance the KV that stays between turns is the cached prefix of each paused session; at a decode instance it is the KV of the running turns.

Definition 4.10 (M/G/∞)

An M/G/∞ system is a delay station with Poisson arrivals of rate λ (Definition 1.17) and i.i.d. service times S independent of the arrival process.

The following lemma is a standard property of the Poisson process.

Lemma 4.11 (Conditional uniformity [20, §2.3])

For a Poisson process of rate λ , the number of arrivals in an interval $(a, b]$ is $\text{Poisson}(\lambda(b-a))$, and conditional on this number being n , the arrival times (as an unordered set) have the law of n i.i.d. samples from the uniform distribution on $(a, b]$.

Proposition 4.12 (M/G/∞ occupancy [20])

If $\mathbb{E}[S] < \infty$, then in an M/G/∞ system started empty at time $-t$, the number of customers at time 0 is $\text{Poisson}(\lambda \int_0^t \mathbb{P}(S > u) du)$, and as $t \rightarrow \infty$ it converges in distribution to $\text{Poisson}(\lambda \mathbb{E}[S])$.

Proof. Let A be the number of arrivals in $(-t, 0]$. Conditional on $A = n$, the arrival times are i.i.d. uniform (Lemma 4.11) and the service times are independent of them. A customer who arrived at time $-u$ is still present at time 0 only if $S > u$, so each customer remains independently with probability $p_t = \frac{1}{t} \int_0^t \mathbb{P}(S > u) du$. Hence the number X remaining satisfies

$$\mathbb{P}(X = k) = \sum_{n \geq k} e^{-\lambda t} \frac{(\lambda t)^n}{n!} \binom{n}{k} p_t^k (1 - p_t)^{n-k} = e^{-\lambda t p_t} \frac{(\lambda t p_t)^k}{k!}.$$

Finally $\lambda t p_t = \lambda \int_0^t \mathbb{P}(S > u) du \rightarrow \lambda \int_0^\infty \mathbb{P}(S > u) du = \lambda \mathbb{E}[S]$. $\square$

That the limit depends on the law of S only through its mean is the insensitivity of the delay station. When the quantity each customer holds (for example, KV bytes) changes over time, the next theorem gives its mean.

Theorem 4.13 (Campbell's mean formula [10])

Let $\{T_k\}$ be the points of a Poisson process of rate λ on $\mathbb{R}$, and attach to each point an i.i.d. mark $Y_k \sim Y$ (with values in an arbitrary measurable space), independent of the points. For measurable $f \geq 0$,

$$\mathbb{E}\left[\sum_k f(T_k, Y_k)\right] = \lambda \int_{\mathbb{R}} \mathbb{E}[f(t, Y)] dt.$$

Proof. First consider the sum over the points in a bounded interval $I = (a, b]$ only. Conditional on the number of points being n , the pairs (T_k, Y_k) are n i.i.d. copies of (U, Y) with $U \sim \text{Uniform}(I)$ and Y independent (Lemma 4.11). Hence

$$\mathbb{E}\left[\sum_{T_k \in I} f(T_k, Y_k)\right] = \sum_{n \geq 0} \mathbb{P}(A_I = n) n \mathbb{E}[f(U, Y)] = \lambda(b-a) \cdot \frac{1}{b-a} \int_a^b \mathbb{E}[f(t, Y)] dt.$$

(The second equality uses Tonelli's theorem.) For $I_m = (-m, m]$ both sides are nondecreasing in m , and the monotone convergence theorem gives the claim as $m \rightarrow \infty$. $\square$

Corollary 4.14 (Mean resident KV)

Let sessions arrive in a Poisson process of rate Λ , and let session k have an i.i.d. lifetime $T_s^{(k)}$ and KV size $K_k(u) \geq 0$ at age u (the lifetime and the trajectory may be dependent). In steady state (started at time $-\infty$), the mean total resident KV at a given instant is

$$\Lambda \mathbb{E}\left[\int_0^{T_s} K(u) du\right].$$

In particular, if $K(u) \equiv \bar{K}$ is constant for each session, this equals $\Lambda(\mathbb{E}[T_s]\mathbb{E}[\bar{K}] + \text{Cov}(T_s, \bar{K}))$. It exceeds the naive product $\Lambda\mathbb{E}[T_s]\mathbb{E}[\bar{K}]$ if and only if $\text{Cov}(T_s, \bar{K}) > 0$, that is, when long sessions have large contexts.

Proof. Attach the mark $Y_k = (T_s^{(k)}, K_k(\cdot))$ and put $f(t, (T, K)) = K(-t)\mathbf{1}\{0 \leq -t < T\}$. The resident KV at time 0 is $\sum_k f(T_k, Y_k)$. By Theorem 4.13 and Tonelli, its mean is

$$\Lambda \int_{-\infty}^0 \mathbb{E}[K(-t)\mathbf{1}\{-t < T_s\}] dt = \Lambda \mathbb{E}\left[\int_0^{T_s} K(u) du\right].$$

If $K \equiv \bar{K}$, the integral is $T_s \bar{K}$, and $\mathbb{E}[T_s \bar{K}] = \mathbb{E}[T_s]\mathbb{E}[\bar{K}] + \text{Cov}(T_s, \bar{K})$. $\square$

Taking $K \equiv 1$ gives the mean number of sessions $\Lambda\mathbb{E}[T_s]$, in agreement with Little's law.

4.3 Eviction: the covering knapsack problem

When the prefill instance's pool runs short, cached prefixes of paused programs must be dropped (**eviction**); a decode instance that keeps paused KV faces the same problem with its own prices (Lecture 5). Program i occupies $c_i > 0$ bytes, and dropping it costs $w_i \geq 0$ in expectation (e.g. p_i times the price of a miss, Theorem 3.10).

Definition 4.15 (Covering knapsack problem)

For a finite set I and a target $\Delta C > 0$, the **covering knapsack problem** is

$$\min_{S \subseteq I} w(S) := \sum_{i \in S} w_i \quad \text{s.t.} \quad c(S) := \sum_{i \in S} c_i \geq \Delta C.$$

Allowing $x \in [0, 1]^I$ in place of S and minimising $\sum w_i x_i$ subject to $\sum c_i x_i \geq \Delta C$ gives the **fractional version**, close to what engines solve, since they drop fixed-size blocks; $v_i = w_i/c_i$ is the **price per byte**.

Theorem 4.16 (Threshold rule [4])

- (i) For every $\theta \geq 0$, the set $T_\theta = \{i : v_i \leq \theta\}$ costs no more than any S with $c(S) \geq c(T_\theta)$.
- (ii) In the fractional version, the solution that sets $x_i = 1$ in increasing order of v_i and fills the last item partially to meet the target exactly is optimal (provided $c(I) \geq \Delta C$).

Proof. (i) On $S \setminus T_\theta$ we have $w_i > \theta c_i$, and on $T_\theta \setminus S$ we have $w_i \leq \theta c_i$, so

$$w(S) - w(T_\theta) = \sum_{S \setminus T_\theta} w_i - \sum_{T_\theta \setminus S} w_i \geq \theta(c(S) - c(T_\theta)) \geq 0.$$

(ii) follows by the same exchange argument with θ the price per byte of the last, partially taken item. $\square$

When programs must be dropped whole, price-per-byte order can be arbitrarily bad, and the standard repair (a greedy guarded by a guess of the most expensive dropped item) is within a factor 2 of the optimum [3]. Real systems evict by a key fixed in advance rather than by a price, such as shortest context first (SF), least recently used (LRU) or a time-to-live; such keys do not look at the resume probability p_i .

Proposition 4.17 (Price-blind keys)

Consider the covering knapsack problem with $w_i = p_i \Phi_i$.

- (i) For every rule that chooses one of two programs from any information except their resume probabilities, and every R , there are resume probabilities in $(0, 1]$ under which either program alone meets the target and the program the rule chooses costs more than R times the other: no such rule has a constant approximation ratio.
- (ii) If every program resumes and $w_i = c_i^2$ (the quadratic term of the recompute work), SF is price-per-byte order and costs at most twice the optimum; the factor 2 is tight (sizes K and $K + 1$ with target $K + 1$).

Proof of (i). Take two programs of the same size c that agree in everything the rule observes, and the target $\Delta C = c$. The rule drops a fixed one, e , whatever the resume probabilities; call the other o . With equal prices Φ , $p_e = 1$ and $p_o = 1/(|R| + 2)$, $w_e = \Phi > R w_o$, while the optimum is at most w_o . $\square$

Even under (ii), SF is not optimal: with contexts 4, 5, 6 and target 6 it drops $\{4, 5\}$ at cost 41, while $\{6\}$ costs 36.

4.4 Exercises

Exercise 4.1. For $N = 2$ and $c = \mu Z$, compute $\sum_n n \pi_2(n)$ directly from (4.1) and check that it equals Q_2 of Theorem 4.5. *Hint:* $G_2 = 1 + 2/c + 2/c^2$.

Exercise 4.2. Show that the stationary mean number of customers in $M/G/\infty$ is $\lambda \mathbb{E}[S]$ without using Proposition 4.12. *Hint:* at a delay station $W = \mathbb{E}[S]$, so use Little's law.

Lecture 5

Putting It Together: A Queueing Model of Disaggregated Serving

The first four lectures built tools. This lecture assembles them into a model of a disaggregated deployment and derives what a scheduler for it needs: what a miss costs at each place where it can happen, what each instance should keep in memory and for how long, how many sessions to admit, and when misses feed themselves. Every step names the result of Lectures 1–4 that carries it.

5.1 The model on one page

Figure 1.1 in Section 1.2 shows the deployment, and Definitions 1.10 and 1.11 and the program of Example 1.12 define the system exactly. We recall the notation.

Definition 5.1 (Disaggregated replica)

A **disaggregated replica** consists of the following stations, visited in this order by every turn.

- (a) A *prefill instance* with a memory pool of M_P bytes: a waiting turn is admitted, in arrival order, only when its whole context fits; the part of the pool not held by admitted turns keeps a *prefix cache* of paused sessions, evicted least recently used first; admitted turns are prefilled one at a time, in arrival order, by a server of full capacity.
- (b) A *link* that carries the turn's KV to a decode instance, shared equally by concurrent transfers: a processor-sharing station of capacity 1 (Definition 2.11).
- (c) A *decode instance* with a pool of M_D bytes, the same whole-context admission, and a processor-sharing batch of capacity $\varphi(m)$ for m running turns.
- (d) The *tool call*, a delay station (Definition 4.1) with mean think time Z , after which the paused session sends its next turn with probability p .

Costs. Prefilling n new tokens on a cached prefix of K tokens takes

$$P(n, K) = a n + b n (K + n/2) \quad (5.1)$$

seconds. A **hit** (the session's prefix is cached) costs $S^{\text{hit}} = P(n, K)$; a **miss** costs $S^{\text{miss}} = P(K + n, 0)$, so the extra work of a miss is $\Delta S = aK + bK^2/2$ (Corollary 3.13). We write S for the prefill work of a turn, h for the hit rate and $\rho_P = \lambda \mathbb{E}[S]$ for the prefill load, with λ the turn rate. A transfer of T_i tokens needs

$$X_i = x_0 + \frac{\kappa T_i}{B} \quad (5.2)$$

seconds of the link, with x_0 a fixed overhead, κ the KV bytes per token and B the bandwidth; the link load is $\rho_L = \lambda \mathbb{E}[X]$. Decoding o_i tokens is a demand D_i at the decode batch. If every context needs about c bytes, instance $j \in \{P, D\}$ has $m_j = \lfloor M_j/c \rfloor$ memory slots, and its admission station is the multi-server queue of Proposition 2.10 with service time equal to the holding time (prefill plus transfer at the prefill instance, decode at the decode instance).

What the time to first token contains. The first token is produced by the prefill instance, so

$$\text{TTFT} = W_P^M + W_P + S,$$

the wait for a memory slot at the prefill instance, the wait for the prefill server, and the prefill itself. The transfer and the decode do not enter it: they delay the second token and every later one. They matter for the TTFT only through memory, since a slot at the prefill instance stays held until the transfer has finished.

5.2 Where the variance of prefill work comes from

The Pollaczek–Khinchine formula makes the second moment of S a first-class quantity (Corollary 3.5). The law of total variance splits it into a part the scheduler does not control and a part it does.

Lemma 5.2 (Variance split of the hit/miss mixture)

Let S_h be the mixture of Definition 3.7: S^{hit} with probability h and S^{miss} with probability $1 - h$, with means $\mu_{\text{hit}}, \mu_{\text{miss}}$ and variances $\sigma_{\text{hit}}^2, \sigma_{\text{miss}}^2$. Then

$$\text{Var}[S_h] = \underbrace{h\sigma_{\text{hit}}^2 + (1-h)\sigma_{\text{miss}}^2}_{\text{spread of the appends}} + \underbrace{h(1-h)(\mu_{\text{miss}} - \mu_{\text{hit}})^2}_{\text{hit/miss mixture}}.$$

Proof. Let I be the indicator of a hit. By the law of total variance, $\text{Var}[S_h] = \mathbb{E}[\text{Var}(S_h | I)] + \text{Var}(\mathbb{E}[S_h | I])$. The first term is $h\sigma_{\text{hit}}^2 + (1-h)\sigma_{\text{miss}}^2$. The conditional mean $\mathbb{E}[S_h | I]$ takes the value μ_{hit} with probability h and μ_{miss} with probability $1 - h$, a two-point variable whose variance is $h(1-h)(\mu_{\text{miss}} - \mu_{\text{hit}})^2$. $\square$

The first term comes from the spread of tool outputs (a line or a whole file) and is not the scheduler's to change. The second is the KV policy's lever: it vanishes at $h \in \{0, 1\}$ and grows with the squared gap between miss and hit work, which by (5.1) grows like K^4 once attention dominates. What a scheduler must price is therefore not a ratio of variances but the rise of $\lambda \mathbb{E}[S^2]/(2(1 - \rho_P))$ that one more miss causes.

5.3 Three prices

A decision that changes where a paused session's KV lives changes the work of its next turn at one of three stations.

(a) A miss at the prefill instance. The prefill instance is an M/G/1 FIFO queue with the whole server available to prefill, so Theorem 3.10 applies as it stands: turning a fraction q_i of arrivals from hits into misses raises the mean number at the prefill instance by an amount in

$[\lambda \sum_i q_i \Phi_i, \frac{1-\rho_P}{1-\rho'_P} \lambda \sum_i q_i \Phi_i]$, with

$$\Phi_i = \Delta S_i + \frac{\lambda((S_i^{\text{miss}})^2 - (S_i^{\text{hit}})^2)}{2(1 - \rho_P)} + \frac{\lambda W_P \Delta S_i}{1 - \rho_P}.$$

The middle term is the head-of-line term: every turn that arrives while the miss is prefilled waits for all of it.

(b) A miss at the decode instance. If the decode instance still holds the session's KV from its previous turn, the next turn transfers only its new tokens, $T_i = n_i$; if not, it transfers the whole context, $T_i = K_i + n_i$. A decode-side miss therefore adds $\Delta X_i = \kappa K_i/B$ of link work.

Proposition 5.3 (The price of a transfer miss)

Let the link be a processor-sharing station of capacity 1 with load $\rho_L < 1$, and turn a fraction q_i of arrivals into decode-side misses, raising the load to $\rho'_L = \rho_L + \lambda \sum_i q_i \Delta X_i < 1$. Then the mean number of transfers in progress rises exactly by

$$\Delta L_L = \frac{1 - \rho_L}{1 - \rho'_L} \lambda \sum_i q_i \Phi_i^X, \quad \Phi_i^X = \frac{\Delta X_i}{(1 - \rho_L)^2},$$

whatever the distribution of the transfer times.

Proof. By insensitivity (Theorem 2.16) the number of transfers in progress has the geometric law of Proposition 2.18 with capacity $C = 1$, for every transfer-time distribution with the given mean. Proposition 2.18 with $C = 1$ and ΔD_i replaced by ΔX_i gives the formula. $\square$

(c) Decode demand. A decision that adds decode demand, such as admitting a session, pays the load factor of Proposition 2.18 at the decode batch; a miss at either instance adds none (Corollary 2.19).

Comparison. The same context of K tokens costs the prefill instance a miss whose work is quadratic in K and whose price has a head-of-line term, set by the second moment of the prefill work; it costs the link a transfer linear in K whose price depends on its mean only, because under processor sharing a long transfer does not block the short ones. The prefix cache of the prefill instance is where memory buys the most.

5.4 The decision problem with two pools

At a decision epoch each instance holds KV for a set of paused sessions. Session i holds c_i bytes, resumes with probability p_i after an expected further τ_i seconds, and its next turn pays Φ_i if its prefix is not in the prefill instance's cache and Φ_i^X if its KV is not at the decode instance. With indicators $x_i^P, x_i^D \in \{0, 1\}$ of “keep” at each instance and byte-second budgets B_P and B_D , the keep-or-drop problem is

$$\text{OPT} = \min_{x^P, x^D} \sum_i [(1 - x_i^P) p_i \Phi_i + (1 - x_i^D) p_i \Phi_i^X] \quad \text{s.t.} \quad \sum_i c_i \tau_i x_i^P \leq B_P, \quad \sum_i c_i \tau_i x_i^D \leq B_D. \quad (5.3)$$

Relaxing the two constraints with multipliers $\theta_P, \theta_D \geq 0$, the **shadow prices** of a byte-second at each instance, gives the Lagrangian $\mathcal{L} = \sum_i [(1 - x_i^P) p_i \Phi_i + (1 - x_i^D) p_i \Phi_i^X] + \theta_P (\sum_i c_i \tau_i x_i^P - B_P) + \theta_D (\sum_i c_i \tau_i x_i^D - B_D)$.

Proposition 5.4 (One price per pool)

Let $u_i^P = p_i \Phi_i / (c_i \tau_i)$ and $u_i^X = p_i \Phi_i^X / (c_i \tau_i)$.

- (i) For all $\theta_P, \theta_D \geq 0$, $\mathcal{L}$ is minimised by keeping session i at the prefill instance iff $u_i^P > \theta_P$ and at the decode instance iff $u_i^X > \theta_D$, and the minimum $g(\theta_P, \theta_D)$ is at most OPT.
- (ii) If the multipliers are such that these choices use both budgets exactly, the choices are optimal for (5.3).

Proof. (i) $\mathcal{L} = \sum_i p_i (\Phi_i + \Phi_i^X) - \theta_P B_P - \theta_D B_D + \sum_i x_i^P (\theta_P c_i \tau_i - p_i \Phi_i) + \sum_i x_i^D (\theta_D c_i \tau_i - p_i \Phi_i^X)$ is separable: each x_i^P is best set to 1 exactly when $\theta_P c_i \tau_i < p_i \Phi_i$, that is $u_i^P > \theta_P$, and likewise each x_i^D . For a feasible (x^P, x^D) both brackets multiplying θ_P and θ_D are nonpositive, so its objective is at least its Lagrangian, which is at least g ; minimising over feasible choices gives $\text{OPT} \geq g$ (weak duality). (ii) The choices of (i) are then feasible and their constraint terms vanish, so their objective equals $g \leq \text{OPT}$. $\square$

This is the threshold rule of Theorem 4.16, once per pool: each instance keeps what is worth more per byte-second than its own memory.

Corollary 5.5 (Free transfer)

As $B \rightarrow \infty$, $\Phi_i^X \rightarrow 0$ for every session, so for any $\theta_D > 0$ the decode instance keeps nothing for paused sessions, and all of M_D serves running decodes.

Proof. $\Delta X_i = \kappa K_i / B \rightarrow 0$, and $\rho_L \rightarrow \lambda x_0$ stays below 1 when $\lambda x_0 < 1$, so $\Phi_i^X = \Delta X_i / (1 - \rho_L)^2 \rightarrow 0$ and $u_i^X \rightarrow 0 < \theta_D$. $\square$

A decode instance's cache is worth exactly the transfers it saves; the prefill instance's cache is worth the recomputation it saves and the queueing that recomputation would cause.

5.5 How long to keep

Proposition 5.4 decides at one instant. Because the Lagrangian is separable, once the multiplier of an instance is fixed each paused session can also be treated on its own over time, at each instance separately, with that instance's price Φ (prefill: Φ_i ; decode: Φ_i^X) and multiplier θ (θ_P or θ_D): keep its state until some time T and drop it then if the program has not resumed. Let the program resume with probability $p \in (0, 1]$, and, given that it resumes, after a time with a continuous distribution function G and density g on $[0, \infty)$. Holding the state costs θc per second; dropping it costs Φ if the program resumes later. The expected cost of the rule “hold until T ” is

$$C(T) = \theta c \int_0^T (1 - pG(t)) dt + \Phi p (1 - G(T)), \quad (5.4)$$

since the state is held at time t exactly when the program has not resumed by t , and a miss occurs exactly when it resumes after T . The quantity that decides is the **resume hazard**

$$\eta(t) = \frac{p g(t)}{1 - pG(t)},$$

the rate at which a program that has not resumed by t resumes at t .

Proposition 5.6 (Optimal holding time)

Let $r = \theta c / \Phi$ with $\Phi > 0$, and suppose g is continuous.

- (i) $C'(T) = (1 - pG(T))(\theta c - \Phi \eta(T))$ for every $T \geq 0$. Holding a little longer lowers the cost exactly when $\Phi \eta(T) > \theta c$.
- (ii) If η is nonincreasing, let $T^* = \inf\{T \geq 0 : \eta(T) \leq r\}$. If $T^* < \infty$ it minimises C over $[0, \infty)$; if $T^* = \infty$, C is nonincreasing and the state should be held until the program resumes. In particular $T^* = 0$, drop at once, if and only if $\eta(0) \leq r$.
- (iii) If G is exponential with mean τ , then η is nonincreasing and $\eta(0) = p/\tau$, so the state should be kept at all if and only if

$$u = \frac{p\Phi}{c\tau} > \theta,$$

the price per byte-second of Proposition 5.4. If $p = 1$ and $u > \theta$, the state is held until the program resumes. If $p < 1$ and $u > \theta$, the optimal holding time is

$$T^* = \tau \ln \left[\frac{p}{1-p} \left(\frac{\Phi}{\theta c \tau} - 1 \right) \right].$$

Proof. (i) The integrand in (5.4) is continuous, so by the fundamental theorem of calculus the first term has derivative $\theta c(1 - pG(T))$, and the second has derivative $-\Phi p g(T)$. Factoring out $1 - pG(T)$ gives the formula when $pG(T) < 1$; when $pG(T) = 1$ (possible only if $p = 1$) both sides vanish, because $g(T) = 0$ wherever G has reached 1 and g is continuous. The factor $1 - pG(T)$ is nonnegative, so the sign of $C'(T)$ is the sign of $\theta c - \Phi \eta(T)$.

(ii) For $T < T^*$, $\eta(T) > r$, so $C'(T) \leq 0$ by (i). For $T > T^*$, $\eta(T) \leq r$ because η is nonincreasing, so $C'(T) \geq 0$. Hence C is nonincreasing on $[0, T^*]$ and nondecreasing on $[T^*, \infty)$, and T^* is a minimiser. If $T^* = \infty$, then $C' \leq 0$ everywhere. The last claim is the definition of T^* at $T = 0$, using that η is nonincreasing.

(iii) With $x = e^{-T/\tau}$, $g(T) = x/\tau$ and $1 - pG(T) = 1 - p + px$, so

$$\eta(T) = \frac{p}{\tau} \cdot \frac{x}{1 - p + px}.$$

The map $x \mapsto x/(1 - p + px)$ is nondecreasing on $(0, 1]$, and x decreases in T , so η is nonincreasing; it is the constant $1/\tau$ when $p = 1$. At $T = 0$, $\eta(0) = p/\tau$, and $\eta(0) > r$ is $p\Phi/(c\tau) > \theta$. If $p = 1$, $\eta \equiv 1/\tau > r$, so $T^* = \infty$ by (ii). If $p < 1$, $\eta(T) \rightarrow 0$ as $T \rightarrow \infty$, so T^* is finite and solves $\eta(T^*) = r$ by continuity: $px/\tau = r(1 - p + px)$, that is $x = r(1 - p)/(p(1/\tau - r))$, and $T^* = -\tau \ln x = \tau \ln \left[\frac{p}{1-p} \left(\frac{1}{r\tau} - 1 \right) \right]$ with $1/(r\tau) = \Phi/(\theta c \tau)$. $\square$

So the instantaneous test $u_i > \theta$ of Proposition 5.4 is exactly the optimal decision to keep a state for exponential resume times, and part (iii) adds how long to keep it. For example, with $p = 0.5$, $\tau = 10$ s, $\Phi = 20$ s, $c = 1$ and $\theta = 0.3$, $u = 1 > \theta$, the optimal holding time is $T^* \approx 17.3$ s, and it lowers the expected cost from $C(0) = p\Phi = 10$ to $C(T^*) \approx 5.6$. When the gap distribution has a decreasing hazard, as heavy-tailed gaps typically do, part (ii) gives a program-specific time-to-live.

Remark 5.7 (Unknown resume times and the oracle)

If nothing is known about G , holding the state is the classical *ski rental* problem: pay rent θc per second or buy once at price Φ [6]. The rule “hold for $\Phi/(\theta c)$ seconds” costs at most twice the cost in hindsight for any resume time: if the program resumes at $\tau \leq \Phi/(\theta c)$ it pays $\theta c\tau$, which is optimal; otherwise it pays 2Φ against a hindsight optimum of Φ . The hindsight optimum, which knows whether and when each program resumes, is called the **oracle**; it keeps a state exactly when the program resumes and $\theta c\tau < \Phi$. It cannot be run online, but it is the benchmark against which a policy’s cost is reported as cost/OPT . The factor two needs the program to resume: against a program that never does, the oracle pays nothing, and no rule that holds the state for a positive time has a bounded ratio.

Proposition 5.6 holds θ and Φ fixed. In the real system both move with the policy: what is dropped changes the load, and the load changes every Φ_i and the value of θ that fills the pool. The joint optimum is a Markov decision problem over the whole state, with no closed form; Proposition 5.6 is its solution at a fixed point of (θ, Φ) .

5.6 How many sessions to admit

Two memory quantities decide admission. The running turns at each instance occupy slots: with exponential holding times the wait for a slot is the Erlang C wait of Proposition 2.10, and it grows without bound as the offered load of slots approaches m_j . The paused sessions occupy the prefix cache: with sessions arriving as a Poisson process of rate Λ , living for T_s and holding $K(t)$ tokens at age t , the mean resident KV is $\kappa\Lambda\mathbb{E}[\int_0^{T_s} K(t) dt]$ (Corollary 4.14), which exceeds the naive product whenever long contexts belong to long sessions. The admission step caps the live sessions near the number at which this estimate fills the prefix cache, and holds the rest in an entry queue. The cap has a price of its own, the wait at the entry queue: a session is worth admitting while the memory it will hold, priced at θ_P per byte-second, costs less than that wait.

5.7 Misses that feed themselves

So far the hit rate was an input. It is also an output. A miss re-inserts a whole context into the prefill instance’s pool, which evicts other paused prefixes sooner (*pool channel*); and a turn that waits longer is absent longer, which exposes its own prefix to eviction (*wait channel*). Let $W(h)$ be the mean wait at the prefill instance at hit rate h , nonincreasing in h (Corollary 3.8), and let $G(T; h)$ be the probability that a paused prefix survives an absence T when the instance runs at hit rate h , nonincreasing in T and nondecreasing in h .

Definition 5.8 (Feedback map)

The **feedback map** is $H(h) = \mathbb{E}_Z[G(Z + W(h); h)]$ on $[0, 1]$, and an **equilibrium** is a hit rate with $H(h) = h$.

The wait channel is an admission rule. Whether a waiting turn’s prefix can be evicted is decided by the engine, not by the workload. An engine that takes the prefix out of the cache only when it schedules the turn (vLLM admits a waiting request at the start of an iteration with the budget left, and until then its cached blocks stay evictable) has the wait channel. An engine that pins the prefix when the turn arrives has survival $G(Z; h)$ instead of $G(Z + W(h); h)$, a larger value for every h , but it holds the pinned bytes and so lowers G for the other paused prefixes; the

comparison of Proposition 5.9(ii) applies only when the first effect dominates. In a simulation of the vLLM engine replaying the short-context agent trace on the A100 pool of 128k tokens, pinning moved the cliff: at one session every 3s the vLLM rule collapsed (full-hit rate 0.24) and the pinning rule did not (0.80); at 3.5s and above the two differed by less than two points.

H is nondecreasing: if $h \leq h'$ then $W(h) \geq W(h')$, so $G(Z + W(h); h) \leq G(Z + W(h'); h) \leq G(Z + W(h'); h')$ for every Z , and taking expectations preserves the order. No continuity is needed for what follows.

Proposition 5.9 (Extremal equilibria)

Let $H : [0, 1] \rightarrow [0, 1]$ be nondecreasing.

- (i) $\bar{h} = \sup\{x : x \leq H(x)\}$ is an equilibrium, and every equilibrium is at most $\bar{h}$; $\underline{h} = \inf\{x : H(x) \leq x\}$ is an equilibrium, and every equilibrium is at least $\underline{h}$.
- (ii) If $K : [0, 1] \rightarrow [0, 1]$ is nondecreasing and $H \leq K$ pointwise, then $\bar{h}_H \leq \bar{h}_K$ and $\underline{h}_H \leq \underline{h}_K$.

Proof. (i) Let $A = \{x \in [0, 1] : x \leq H(x)\}$; $0 \in A$, so $\bar{h} = \sup A$ exists. For $x \in A$, $x \leq H(x) \leq H(\bar{h})$ because $x \leq \bar{h}$ and H is nondecreasing; so $H(\bar{h})$ is an upper bound of A and $\bar{h} \leq H(\bar{h})$. Then $H(\bar{h}) \leq H(H(\bar{h}))$, so $H(\bar{h}) \in A$ and $H(\bar{h}) \leq \bar{h}$. Hence $H(\bar{h}) = \bar{h}$. Every equilibrium lies in A , so it is at most $\bar{h}$. The argument for $\underline{h}$ is the mirror image with $B = \{x : H(x) \leq x\} \ni 1$ and infima. (ii) $\bar{h}_H = H(\bar{h}_H) \leq K(\bar{h}_H)$, so $\bar{h}_H$ lies in the set A of K , and $\bar{h}_H \leq \bar{h}_K$. Likewise $H(\underline{h}_K) \leq K(\underline{h}_K) = \underline{h}_K$ puts $\underline{h}_K$ in the set B of H , so $\underline{h}_H \leq \underline{h}_K$. $\square$

A larger prefix cache raises G and so H , and by (ii) it raises both extremal equilibria. Other changes need care in a closed loop: a shorter think time or a faster decode also raises the turn rate, and the net effect on H has no sign in general.

Proposition 5.10 (Collapse)

In the open prefill instance take the wait of Theorem 3.4, $W(h) = \infty$ where $\lambda \mathbb{E}[S_h] \geq 1$, and $G(\infty; h) = 0$.

- (i) If $\lambda \mathbb{E}[S^{\text{miss}}] \geq 1$, then $H(0) = 0$, and every h at which the instance is overloaded is mapped to 0, which is mapped to itself: all-miss attracts the overloaded hit rates.
- (ii) If moreover $H(a) \geq a$ for some $a > 0$, the least equilibrium is 0 and the greatest is at least a : the instance has two distinct extremal equilibria.

Proof. (i) At $h = 0$ every turn misses, the load is $\lambda \mathbb{E}[S^{\text{miss}}] \geq 1$, so $W(0) = \infty$ and $H(0) = G(\infty; 0) = 0$. At an overloaded h , $H(h) = 0$ in the same way, and $H(0) = 0$. (ii) 0 is an equilibrium, so $\underline{h} \leq 0$; and $a \in A$, so $\bar{h} \geq a$ by Proposition 5.9. $\square$

In a closed system the population bounds the wait and removes this route to $H(0) = 0$, but not the pool channel: if re-inserted contexts evict every paused prefix within a think time, $H(0) = 0$ still. Thrashing, a collapse of this kind cured by limiting the load, is classical in paging systems; the admission cap of Section 5.6 is its cure here.

Example 5.11 (Two equilibria and a cap)

Take deterministic $S^{\text{hit}} = 0.5\text{s}$, $S^{\text{miss}} = 5\text{s}$, $Z = 7\text{s}$ and the wait channel alone, $G(T) = e^{-T/60\text{s}}$, with the Pollaczek–Khinchine wait. At $\lambda = 0.25/\text{s}$ the all-miss load is 1.25, $H = 0$ on $[0, 2/9)$, and a grid search finds three equilibria: 0 (stable), about 0.250 (unstable, slope $H' \approx 12$) and about 0.882 (stable, slope 0.07). At $\lambda = 0.18/\text{s}$ the all-miss load is 0.9 and the only equilibrium is about 0.885. Lowering the load removes the collapse without moving the good equilibrium much.

Proposition 5.12 (The forced-miss multiplier)

Let $H(h) = c + kh$ near an equilibrium, with $c \geq 0$ and $0 \leq k < 1$, so $h_0 = c/(1 - k)$, and force a fraction $\delta \in [0, 1]$ of turns to miss, so the realised hit rate is $(1 - \delta)H(h)$. The new equilibrium is $h_\delta = (1 - \delta)c/(1 - (1 - \delta)k)$, and

$$h_0 - h_\delta = \frac{\delta c}{(1 - k)(1 - (1 - \delta)k)} \geq \delta h_0.$$

Proof. $h_\delta(1 - (1 - \delta)k) = (1 - \delta)c$ is the equation $h = (1 - \delta)(c + kh)$. Over the common denominator $(1 - k)(1 - (1 - \delta)k)$ the numerator of $h_0 - h_\delta$ is $c(1 - (1 - \delta)k) - (1 - \delta)(1 - k)c = \delta c$. The inequality is $1/(1 - (1 - \delta)k) \geq 1$, true since $(1 - \delta)k \geq 0$. $\square$

To first order in δ a forced miss converts a would-be hit with probability h_0 and induces $h_0 k/(1 - k)$ further misses on other turns, each with its own price, so its marginal price is $h_0[\Phi_{\text{forced}} + \frac{k}{1-k}\bar{\Phi}_{\text{induced}}]$. This cache multiplier multiplies the queue factor $(1 - \rho_P)/(1 - \rho'_P)$ of Theorem 3.10; it does not replace it. A multiplier common to all sessions scales every u_i^P by the same factor: it leaves the eviction order unchanged and moves only the comparisons with θ_P (keep or drop, admit).

5.8 What is exact and what is approximate

- *Exact, given the model:* the price bracket (Theorem 3.10), the prices at processor-sharing stations (Propositions 2.18 and 5.3), the monotonicity of the decode batch (Theorem 2.14), the finite-population inequalities (Proposition 4.6), the price-blind bounds (Proposition 4.17), and Propositions 5.4 to 5.12.
- *Modelling choices:* the Poisson reading of turn arrivals, which the session loop makes non-Poisson (Proposition 4.6 shows the open formula overstates the wait for exponential work); equal slot sizes (with unequal contexts a turn needs several slots and the memory station has no closed form); the link as fair bandwidth sharing (a transfer engine that sends in arrival order would add a head-of-line term to Proposition 5.3); a single survival law and the mean wait in the feedback map; and the pricing of one eviction, a transient event, by a stationary comparison.
- *Estimates:* p_i , τ_i and G must be predicted from a session's phase, tool and history; a , b , x_0 , κ , B and φ must be measured on the instances.

5.9 Exercises

Exercise 5.1. In Lemma 5.2 let $\sigma_{\text{hit}} = \sigma_{\text{miss}} = \sigma$. Find the hit rate that maximises the mixture term, and show that the mixture term exceeds the append term exactly when $h(1 - h)(\mu_{\text{miss}} -$

$\mu_{\text{hit}})^2 > \sigma^2$. *Hint:* $h(1-h)$ is maximised at $h = 1/2$.

Exercise 5.2. A session with context K is paused at both instances. Using Proposition 5.4, show that for K large enough it is kept at the prefill instance whenever it is kept at the decode instance, if $\theta_P = \theta_D$ and $b > 0$. *Hint:* $\Phi_i \geq \Delta S_i \geq bK^2/2$ grows quadratically, Φ_i^X linearly in K .

Bibliography

- [1] S. Asmussen, *Applied Probability and Queues*, 2nd ed., Springer, 2003.
- [2] F. Baskett, K. M. Chandy, R. R. Muntz, F. G. Palacios, Open, closed, and mixed networks of queues with different classes of customers, *Journal of the ACM* 22(2), 1975.
- [3] J. Csirik, J. B. G. Frenk, M. Labbé, S. Zhang, Heuristics for the 0-1 min-knapsack problem, *Acta Cybernetica* 10, 1991.
- [4] G. B. Dantzig, Discrete-variable extremum problems, *Operations Research* 5(2), 1957.
- [5] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*, Cambridge University Press, 2013.
- [6] A. R. Karlin, M. S. Manasse, L. Rudolph, D. D. Sleator, Competitive snoopy caching, *Algorithmica* 3, 1988.
- [7] F. P. Kelly, *Reversibility and Stochastic Networks*, Wiley, 1979.
- [8] A. Khinchine, Mathematical theory of a stationary queue, *Matematicheskii Sbornik*, 1932.
- [9] A. Y. Khinchine, *Mathematical Methods in the Theory of Queueing*, Griffin, London, 1960.
- [10] J. F. C. Kingman, *Poisson Processes*, Oxford University Press, 1993.
- [11] L. Kleinrock, *Queueing Systems, Volume 2: Computer Applications*, Wiley, 1976.
- [12] L. Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley, 1975.
- [13] E. D. Lazowska, J. Zahorjan, G. S. Graham, K. C. Sevcik, *Quantitative System Performance*, Prentice-Hall, 1984.
- [14] J. D. C. Little, A proof for the queueing formula $L = \lambda W$, *Operations Research* 9(3), 1961.
- [15] R. M. Loynes, The stability of a queue with non-independent inter-arrival and service times, *Proc. Cambridge Philosophical Society* 58(3), 1962.
- [16] P. Naor, The regulation of queue size by levying tolls, *Econometrica* 37(1), 1969.
- [17] J. R. Norris, *Markov Chains*, Cambridge University Press, 1997.
- [18] F. Pollaczek, Über eine Aufgabe der Wahrscheinlichkeitstheorie, *Mathematische Zeitschrift*, 1930.
- [19] M. Reiser, S. S. Lavenberg, Mean-value analysis of closed multichain queueing networks, *Journal of the ACM* 27(2), 1980.
- [20] S. M. Ross, *Stochastic Processes*, 2nd ed., Wiley, 1996.
- [21] A. Agrawal et al., Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, 2024.
- [22] S. Stidham, A last word on $L = \lambda W$, *Operations Research* 22(2), 1974.
- [23] R. W. Wolff, Poisson arrivals see time averages, *Operations Research* 30(2), 1982.